

BuildWise: Consolidated SE3090 Report Draft

Group: G07 (existing package identifier; confirm before submission)

Revision date: 2026-10-05. Repository: <https://github.com/IT24102414/build>

Status: evidence-backed technical draft. It is not a signed final submission. The specification deadline was 2026-09-30; this revision does not assert an earlier submission. Individual reflections, signatures, deployment links and video require student input.

Part A: Group report

1. Problem, objectives and scope

Construction projects need a traceable connection between requested materials, supplier offers, purchasing decisions, receiving records and quality outcomes. BuildWise records that lifecycle in one shared system. Operational staff work on mobile; desk users review data and decisions on web. The same operations are available to authorized roles in Flutter so cross-platform access does not change business rules.

The four primary business components are material request/approval, supplier/RFQ/quotation/procurement, delivery/receiving, and quality/NCR management. Suppliers are external contacts approached by email, not authenticated internal users. The seven roles and demo accounts are listed in the root README.

2. Functional requirements and rules

Material requests contain project, material, unit, quantity, priority, reason and dates. Typed material names accept letters and numbers and may use suggestions. Site Engineers create their requests. Site Officers track their own request records and receive deliveries. Approvers review demand; the procurement officer records supplier offers and initiates analysis; the procurement manager records the purchasing decision.

Approval creates a confirmed purchase order through the API transaction. The receiving list refreshes to make that order available. Recording deliveries maintains ordered/received/damaged/shortage context. Inspection materials must belong to the selected delivery and inspected quantities cannot exceed the corresponding received quantities.

Inspection rules: positive inspected quantity; non-negative rejected quantity; rejected no larger than inspected; accepted plus rejected equals inspected; at least one line; notes for failed checklist items; reason for rejected quantities; at most ten evidence files, each at most 10 MB. Rejected quantity zero creates no NCR. A positive rejection automatically creates an NCR. Resolved, Closed and AcceptedException require a resolution. Backend transition validation remains authoritative.

Role navigation follows the React navigation table. Flutter Site Officers use the own-request endpoint rather than the full register. Field/quality users can read purchase orders with commercial values redacted by the API. Camera/gallery evidence and local notifications are device capabilities; hardware permission behavior still requires an installed-device run.

3. Integrated architecture

Public clients -> ASP.NET Core API -> PostgreSQL and internal Python agent services.

React and Flutter call only the ASP.NET API. Authentication uses hashed passwords and JWTs; authorization policies protect operations. EF Core coordinates relational persistence. Controllers accept DTOs, services enforce rules, and middleware handles auditing/error concerns. Python service ports are internal deployment endpoints, not public client integration points.

Layer	Responsibility	Evidence
React	Role routing, forms, dashboards, approval, workflow summaries	web/buildwise-web/src
Flutter	Operational forms, role workspaces, secure session, evidence capture	mobile/buildwise_mobile/lib
API	Identity, business rules, transactions, audits, workflow orchestration	backend/BuildWise.Api
PostgreSQL	Domain entities, relationships, durable workflow state	EF migrations and configurations
Agents	Domain analysis, structured outputs, bounded tools, failure results	backend/agent_service

4. Database design

Projects relate to requests; request lines relate to materials; supplier offers relate to approved requests and quotations; approved procurement produces orders and order items; deliveries relate to orders; inspections relate to deliveries and materials; rejection produces NCR follow-up. Agent workflows relate to their steps, validation outcomes and human approvals. User/role relationships govern access.

Use the EF model and migrations as the authoritative schema. The existing ERD DBML/PDF are supporting project diagrams and should be checked against the final migration set before submission. The evidence URL storage migration supports validated inline image data as well as permitted URLs. Entity DTOs expose only the information the caller's policy allows; commercial terms are redacted for field/quality roles.

PostgreSQL stores execution summaries needed for auditability, not hidden model reasoning. High-impact approval is transactional: the recorded decision and resulting confirmed purchase order must stay consistent. State fields and related-record references let the second client observe the updated outcome.

5. API design and component evidence

Component	Representative endpoints / operations	Validation and role boundary
Requests	GET/POST /api/material-requests; GET /api/material-requests/{id}; POST approval; POST request analysis	Required project/material/quantity; approver-only decisions
Procurement	Suppliers CRUD; RFQ issue/close/email; quotation create/delete/compare; workflow start/latest/decision	Officer entry/analysis; manager approval; eligibility and budget checks
Receiving	GET/POST /api/deliveries; confirmed purchase order read; delivery detail; discrepancy analysis	Authorized receiving; ordered/received/discrepancy business rules
Quality	GET/POST /api/quality-inspections; inspection risk analysis; NCR read/status operations	Delivery/material/quantity/evidence rules; valid resolution transitions

Swagger is the authoritative route/DTO reference at /swagger. The API supplies appropriate success/failure status codes and structured validation errors; clients show field/API errors rather than hiding every failure behind a generic 400.

6. React design

Functional components, hooks, React Router and Context authentication provide routing/session state. Domain service modules centralize API calls. Reusable cards, fields, tables, status badges and

error/loading/empty states serve the business pages. Role-filtered navigation and per-operation capabilities complement server enforcement.

The latest update removes unauthorized supplier prefetch when read-only users open purchase orders. Procurement workspaces load the latest persisted recommendation, making officer and manager sessions observe the same result. Responsive layout and phone-width tables were checked with local Playwright.

7. Flutter design

Flutter uses reusable widgets, StatefulWidget state, injected services for tests and secure token storage. The responsive navy menu, role labels, order and Plus Jakarta Sans typography align with React. Native operational capabilities remain camera/gallery evidence and local notifications.

Request lists refresh; procurement comparison preserves the selected request and reads the latest workflow; officers record/remove eligible offers and run analysis; managers read recommendations, edit allowed budgets and make decisions. Budget values must be non-negative or blank to clear allocation. Dashboard activity refreshes every 15 seconds while resumed and on resume. Public endpoint/status details are available without recording request bodies or secrets.

8. Agentic AI architecture and acceptance workflow

The assessed procurement path receives a domain objective, produces a structured plan and delegates tasks, reads allowed domain tools, ranks eligible offers, applies deterministic validation and pauses before purchase order creation. Authorized approval then produces an auditable result; failed validation or service failure is recorded safely.

Responsibility	Contract / controls	Visible outcome
Planning	Structured procurement objective and read-tool inputs	Plan and delegated steps
Quotation analysis	Active suppliers, offered/requested quantities and totals	Eligible offer ranking and recommendation
Deterministic validation	Schema/business rules and budget context	Validity findings and warnings
Request analysis	Request/project/items context	Planning/urgency flags
Delivery discrepancy	Ordered, received and damaged quantities	Shortage/damage findings
Quality risk	Inspection quantities/reasons	Risk result and NCR recommendation

Agents have different inputs, outputs and responsibilities. The deployed runtime may use optional LLM rationale; deterministic checks and failure behavior remain necessary when provider credentials are absent or invalid. A live golden-case walkthrough must show persisted steps, validated tool results, authorized approval and final PO, not merely a single prompt response.

9. Security and external integration

Secrets belong in environment variables or managed secret storage. Never include .env, provider keys, virtual environments, runtime dumps or request tokens in a submission archive. The updated packager uses git archive of a named committed revision rather than copying local folders indiscriminately.

Self-service API registration permits front-line roles; manager/administrator accounts are provisioned by an administrator. A separate Flutter registration interface was not evidenced in this maintenance run and remains a requirement check. SMTP access is backend-owned. Failed RFQ email is presented as failed

with retry controls; a successful HTTP request alone is not evidence that email was sent. No SMTP provider was configured in this local verification.

10. Software testing report

Suite	Observed result	Scope
.NET/xUnit	307 passed	Services, validation, authorization, controllers and API integration
Python/pytest	28 passed	Four domain agents and deterministic contract assertions
React/Vitest	163 passed	Components, forms, API errors and role behavior
Flutter	149 passed	Widgets, services, quantities, evidence, role scopes and navigation
Final Flutter auth/shell follow-up	10 passed	Required credentials, login transitions, responsive menu/actions
Production builds	React and Flutter web passed	Bundling/compilation evidence

An initial sandboxed .NET run failed on Windows EventLog permissions; the authorized rerun passed. React lint exits successfully but has existing warnings, which must not be described as zero warnings. Browser checks are read-only. Automated fixtures and widget tests do not establish every physical-device or cross-platform write operation.

A complete seven-role browser review made 179 checks and found a React supplier-prefetch permission defect. After fixing it, a targeted seven-role follow-up made 44 checks with no captured errors or page-width overflow. No unsupported Flutter role workspace or missing expected menu was reported.

11. Agent evaluation report

Evaluation combines Python contract tests with API validation and approval tests. Review `test_quotation_agent.py`, `test_request_agent.py`, `test_delivery_agent.py`, `test_quality_agent.py`, `FullLifecycleScenarioTests.cs` and RBAC tests. These exercise domain outputs and boundaries rather than relying solely on an LLM judge.

Before final demonstration, capture a golden procurement case, an ineligible supplier case, invalid quantities, missing revision comments, unauthorized approval, agent unavailability and prompt/tool-input rejection. Record workflow IDs, role, expected result, observed state and test version. Do not claim live provider behavior that was not exercised.

12. Performance report

The performance harness is `scripts/performance-test.mjs`. It measures concurrent API/database reads and each internal agent separately, recording success rates, p50/p95/max latency and throughput. The local output is `docs/performance-results-2026-10-05.json` when the run completes.

These are local Windows results, not a production SLA, network benchmark or separately isolated database latency measurement. Report concurrency, request count, timestamp and environment with the data. Repeat against deployment if production performance is claimed.

13. Deployment and CI report

GitHub Actions builds/tests .NET, Python, React and Flutter and uploads build/APK artifacts. Flutter is pinned to the verified 3.47.5 toolchain. The web API base URL is configurable using the VITE_API_BASE_URL repository variable. The deployment ADR records a proposal, not a verified public service. No successful Actions result should be asserted until the published run is inspected.

Required evaluator evidence: public React URL, API health/Swagger URLs, PostgreSQL host evidence, internal agent access from the API, current installed APK and accessible ten-minute video. The root BuildWise.apk is historical and is excluded from new source archives; supply a current verified APK to the package script. Keep access until at least 2026-10-21.

14. Demonstration sequence

1. Site Engineer creates a validated request in Flutter.
2. Site Manager opens it in React, reviews analysis and approves demand.
3. Procurement Officer records quotations and starts planning/analysis.
4. Show persisted plan, delegated steps, tool results and validation in the workflow history.
5. Procurement Manager reviews, demonstrates a missing revision-comment rejection, then approves a suitable proposal.
6. Show the confirmed purchase order and returned Flutter status.
7. Site Officer records receiving; show discrepancy analysis.
8. Quality Inspector records rejection with notes, reason and evidence; show automatic NCR.
9. Authorized manager resolves the NCR with a resolution; show rejected invalid transitions.
10. Show tests, actual Git/CI evidence and deployment links. Each member explains their component without external AI assistance.

15. Architecture decision records

The report renderer includes the ADR source documents as appendices: durable agent state, orchestration, React state, Flutter state, hosting proposal and external supplier boundaries. These historical decisions include current scope/deployment notes. Students must confirm they understand and own the decisions before signing.

16. Group AI disclosure

This maintenance and report draft were AI-assisted using Codex on 2026-10-05. Outputs and actual verification are recorded in docs/reports/2026-10-05-codex-maintenance-log.md. Existing student AI logs/declarations are historical material that the students must review and update; this report does not certify their completeness.

Consolidated group declaration — UNSIGNED TEMPLATE: We disclose all coding-assistant use in our individual logs and the group maintenance record. Each member has reviewed the work claimed under their name and can explain, test and modify it. Each personal reflection is the student's own writing. Historical claims have been checked against actual evidence. This statement becomes a declaration only after all four members verify and sign it.

Student 1 name/ID/signature/date: [complete after review]. Student 2 name/ID/signature/date: [complete after review]. Student 3 name/ID/signature/date: [complete after review]. Student 4 name/ID/signature/date: [complete after review].

Part B: Individual report sections

The following sections describe repository-supported component work, not verified personal authorship. Each named student must confirm the assigned scope and replace unsigned personal fields. Historical AI logs are included for review; their dates, models, results and reflections are not independently certified by this report.

Student 1: Peiris DPSS

Primary component: A — Material Request and Approval. Student ID and registered name require confirmation.

Contribution statement and technical work

Component responsibility proposed by the existing project ownership map: implement and integrate the API, relational entities, React screens, Flutter screens, tests and domain agent below. Student confirmation: [write the specific work you personally implemented, reviewed and tested; identify shared work and collaborators]. Repository presence alone does not establish who wrote the code.

Demand assessment and human approval beyond CRUD. The API checks required request information, role scope and approval rules; the request agent supports assessment without replacing the approver.

Database scope: MaterialRequest, MaterialRequestItem, Approval. Inspect corresponding EF configurations and migrations for primary/foreign keys, relationships, constraints and audit fields. Explain how the component consumes upstream IDs and exposes downstream records without duplicating another component's source of truth.

Layer	Evidence file
ASP.NET Core API	backend/BuildWise.Api/Controllers/MaterialRequestsController.cs
Backend regression tests	backend/BuildWise.Api.Tests/MaterialRequestServiceTests.cs
React UI	web/buildwise-web/src/pages/MaterialRequestsPage.jsx
Flutter UI	mobile/buildwise_mobile/lib/features/operations/screens/material_requests_screen.dart
Flutter tests	mobile/buildwise_mobile/test/material_requests_test.dart
Distinct domain agent	backend/agent_service/request_agent.py
Agent tests	backend/agent_service/test_request_agent.py

Meaningful API endpoints: GET /api/material-requests; GET /api/material-requests/my; GET /api/material-requests/{id}; POST /api/material-requests; POST /api/material-requests/{id}/approval; POST /api/agent/analyze-request/{id}. Route parameters are symbolic; inspect controller attributes for exact constraints. This supplies at least four component operations; explain authorization, input DTOs, success responses and invalid-input handling in the viva.

Distinct Agentic AI contribution

The component agent is `request_agent.py` with the companion `test_request_agent.py` suite. Document its input contract, permitted context/tool use, structured output and deterministic checks before a human consumes its recommendation. A model-generated rationale is advisory; the API retains business-rule enforcement. Student confirmation: [identify your personally implemented agent function, orchestration/state changes, rejected alternatives and verification]. Explain how a unavailable provider or malformed recommendation is surfaced rather than silently recorded as a valid decision.

Commit, pull-request and test evidence

The following is real path-based Git history, including the recorded author. It must not be relabelled as the named student's authorship. Add actual reviewed PR URLs or state that no PR evidence is available.

```
4e09d1c | Loganathan Theebika | Align React and Flutter workflows and refresh SE3090 submission documentation
c318c46 | Loganathan Theebika | Complete field mobile workspaces: approvals, review, PO, quotation entry, RBAC, validation UX
04d35a6 | Loganathan Theebika | Complete field mobile workspaces
9ab414f | IT24102414 | feat: RBAC hardening, supplier portal, budget validation and C1-C4 coverage
```

Commit URLs use `https://github.com/IT24102414/build/commit/` followed by the recorded hash. Personal commit/PR attribution: [student verifies]. Tests above belong to the observed group suites; component-specific passed counts must come from a recorded focused run, not a guessed division of the group totals.

Challenges and learning — student input

Describe one actual cross-stack contract problem, one invalid input or role boundary you tested, and one agent output you corrected or rejected. Reference the exact change/test. Explain what you can now implement or debug without external AI assistance. Do not claim these experiences until you have personally reviewed and reproduced them.

Individual AI usage log

The existing log `IT24XXXXX-Peiris-ai-usage-log.md` is reproduced in the individual-log appendix under this student's label. Verify each date, tool/model, prompt/task, output, correction and test result; remove unsupported entries. Add the actual 5 October maintenance work you reviewed, distinguishing coding-assistant use from the application's own agents. Do not invent past prompts or dates.

Approximately one-page personal AI reflection — student authored

[Write approximately one page in your own words. Paragraph 1: tools used and stages of work. Paragraph 2: a useful output and a wrong output, tied to real evidence. Paragraph 3: what you changed, added or rejected and why. Paragraph 4: what you learned about your skills and limits, and how you verified ownership. Use your actual log and Git history. This report deliberately does not generate a first-person reflection for submission.]

Signed individual declaration — unsigned template

I confirm that my contribution statement and AI usage log accurately describe my work and assistance received, and that my reflection is my own writing. I can explain, test and modify the work claimed under my

name. I have reviewed the evidence linked in this section.

Registered name: [student completes]. Student ID: [student completes]. Signature: [student signs after review]. Date: [actual signing date].

Student 2: Theebika — IT24102414

Primary component: B — Procurement. Confirm full registered name.

Contribution statement and technical work

Component responsibility proposed by the existing project ownership map: implement and integrate the API, relational entities, React screens, Flutter screens, tests and domain agent below. Student confirmation: [write the specific work you personally implemented, reviewed and tested; identify shared work and collaborators]. Repository presence alone does not establish who wrote the code.

Quotation analysis, budget validation and human procurement decision. Officers enter offers and initiate analysis; managers approve or request changes. Successful approval creates a confirmed purchase order transactionally.

Database scope: Supplier, Rfq, Quotation, PurchaseOrder, AgentWorkflow, AgentWorkflowStep, AgentApproval. Inspect corresponding EF configurations and migrations for primary/foreign keys, relationships, constraints and audit fields. Explain how the component consumes upstream IDs and exposes downstream records without duplicating another component's source of truth.

Layer	Evidence file
ASP.NET Core API	backend/BuildWise.Api/Controllers/ProcurementWorkflowController.cs
Backend regression tests	backend/BuildWise.Api.Tests/ProcurementValidationServiceTests.cs
React UI	web/buildwise-web/src/Features/procurement/pages/RequestWorkspace.jsx; components/QuotationEntryForm.jsx; components/QuotationComparisonView.jsx; components/ProcurementApprovalPanel.jsx
Flutter UI	mobile/buildwise_mobile/lib/features/procurement/screens/quotation_comparison_screen.dart
Flutter tests	mobile/buildwise_mobile/test/quotation_entry_test.dart
Distinct domain agent	backend/agent_service/quotation_agent.py
Agent tests	backend/agent_service/test_quotation_agent.py

Meaningful API endpoints: POST /api/material-requests/{requestId}/procurement-workflow; GET /api/procurement-workflow/{workflowId}; GET /api/procurement-workflow/{workflowId}/history; POST /api/procurement-workflow/{workflowId}/decision. Route parameters are symbolic; inspect controller attributes for exact constraints. This supplies at least four component operations; explain authorization, input DTOs, success responses and invalid-input handling in the viva.

Distinct Agentic AI contribution

The component agent is quotation_agent.py with the companion test_quotation_agent.py suite. Document its input contract, permitted context/tool use, structured output and deterministic checks before a human consumes its recommendation. A model-generated rationale is advisory; the API retains business-rule enforcement. Student confirmation: [identify your personally implemented agent function,

orchestration/state changes, rejected alternatives and verification]. Explain how a unavailable provider or malformed recommendation is surfaced rather than silently recorded as a valid decision.

Commit, pull-request and test evidence

The following is real path-based Git history, including the recorded author. It must not be relabelled as the named student's authorship. Add actual reviewed PR URLs or state that no PR evidence is available.

```
4e09d1c | Loganathan Theebika | Align React and Flutter workflows and refresh SE3090 submission documentation

c318c46 | Loganathan Theebika | Complete field mobile workspaces: approvals, review, PO, quotation entry,
RBAC, validation UX

04d35a6 | Loganathan Theebika | Complete field mobile workspaces

9ab414f | IT24102414 | feat: RBAC hardening, supplier portal, budget validation and C1-C4 coverage
```

Commit URLs use <https://github.com/IT24102414/build/commit/> followed by the recorded hash. Personal commit/PR attribution: [student verifies]. Tests above belong to the observed group suites; component-specific passed counts must come from a recorded focused run, not a guessed division of the group totals.

Challenges and learning — student input

Describe one actual cross-stack contract problem, one invalid input or role boundary you tested, and one agent output you corrected or rejected. Reference the exact change/test. Explain what you can now implement or debug without external AI assistance. Do not claim these experiences until you have personally reviewed and reproduced them.

Individual AI usage log

The existing log `IT24102414-ai-usage-log.md` is reproduced in the individual-log appendix under this student's label. Verify each date, tool/model, prompt/task, output, correction and test result; remove unsupported entries. Add the actual 5 October maintenance work you reviewed, distinguishing coding-assistant use from the application's own agents. Do not invent past prompts or dates.

Approximately one-page personal AI reflection — student authored

[Write approximately one page in your own words. Paragraph 1: tools used and stages of work. Paragraph 2: a useful output and a wrong output, tied to real evidence. Paragraph 3: what you changed, added or rejected and why. Paragraph 4: what you learned about your skills and limits, and how you verified ownership. Use your actual log and Git history. This report deliberately does not generate a first-person reflection for submission.]

Signed individual declaration — unsigned template

I confirm that my contribution statement and AI usage log accurately describe my work and assistance received, and that my reflection is my own writing. I can explain, test and modify the work claimed under my name. I have reviewed the evidence linked in this section.

Registered name: [student completes]. Student ID: [student completes]. Signature: [student signs after review]. Date: [actual signing date].

Student 3: Ramya — IT24102513

Primary component: C — Delivery and Receiving. Confirm full registered name.

Contribution statement and technical work

Component responsibility proposed by the existing project ownership map: implement and integrate the API, relational entities, React screens, Flutter screens, tests and domain agent below. Student confirmation: [write the specific work you personally implemented, reviewed and tested; identify shared work and collaborators]. Repository presence alone does not establish who wrote the code.

Receiving and discrepancy classification beyond CRUD. Confirmed orders feed receiving; persisted received quantities establish the upper bound for downstream inspection. Supplier/material labels must use the actual DTO shape.

Database scope: Delivery, DeliveryItem, DeliveryIssue, DeliveryEvidence. Inspect corresponding EF configurations and migrations for primary/foreign keys, relationships, constraints and audit fields. Explain how the component consumes upstream IDs and exposes downstream records without duplicating another component's source of truth.

Layer	Evidence file
ASP.NET Core API	backend/BuildWise.Api/Controllers/DeliveriesController.cs
Backend regression tests	backend/BuildWise.Api.Tests/DeliveryServiceTests.cs
React UI	web/buildwise-web/src/pages/DeliveriesPage.jsx
Flutter UI	mobile/buildwise_mobile/lib/features/operations/screens/delivery_receiving_screen.dart
Flutter tests	mobile/buildwise_mobile/test/delivery_receiving_test.dart
Distinct domain agent	backend/agent_service/delivery_agent.py
Agent tests	backend/agent_service/test_delivery_agent.py

Meaningful API endpoints: GET /api/deliveries; GET /api/deliveries/confirmed-orders; POST /api/deliveries; POST /api/deliveries/{id}/discrepancy-analysis; GET /api/deliveries/{id}/issues. Route parameters are symbolic; inspect controller attributes for exact constraints. This supplies at least four component operations; explain authorization, input DTOs, success responses and invalid-input handling in the viva.

Distinct Agentic AI contribution

The component agent is `delivery_agent.py` with the companion `test_delivery_agent.py` suite. Document its input contract, permitted context/tool use, structured output and deterministic checks before a human consumes its recommendation. A model-generated rationale is advisory; the API retains business-rule enforcement. Student confirmation: [identify your personally implemented agent function, orchestration/state changes, rejected alternatives and verification]. Explain how a unavailable provider or malformed recommendation is surfaced rather than silently recorded as a valid decision.

Commit, pull-request and test evidence

The following is real path-based Git history, including the recorded author. It must not be relabelled as the named student's authorship. Add actual reviewed PR URLs or state that no PR evidence is available.

4e09d1c | Loganathan Theebika | Align React and Flutter workflows and refresh SE3090 submission documentation

04d35a6 | Loganathan Theebika | Complete field mobile workspaces

9ab414f | IT24102414 | feat: RBAC hardening, supplier portal, budget validation and C1-C4 coverage

9466c5e | Loganathan Theebika | feat: complete procurement and operations workflow

Commit URLs use <https://github.com/IT24102414/build/commit/> followed by the recorded hash. Personal commit/PR attribution: [student verifies]. Tests above belong to the observed group suites; component-specific passed counts must come from a recorded focused run, not a guessed division of the group totals.

Challenges and learning — student input

Describe one actual cross-stack contract problem, one invalid input or role boundary you tested, and one agent output you corrected or rejected. Reference the exact change/test. Explain what you can now implement or debug without external AI assistance. Do not claim these experiences until you have personally reviewed and reproduced them.

Individual AI usage log

The existing log IT24102513-Ramya-ai-usage-log.md is reproduced in the individual-log appendix under this student's label. Verify each date, tool/model, prompt/task, output, correction and test result; remove unsupported entries. Add the actual 5 October maintenance work you reviewed, distinguishing coding-assistant use from the application's own agents. Do not invent past prompts or dates.

Approximately one-page personal AI reflection — student authored

[Write approximately one page in your own words. Paragraph 1: tools used and stages of work. Paragraph 2: a useful output and a wrong output, tied to real evidence. Paragraph 3: what you changed, added or rejected and why. Paragraph 4: what you learned about your skills and limits, and how you verified ownership. Use your actual log and Git history. This report deliberately does not generate a first-person reflection for submission.]

Signed individual declaration — unsigned template

I confirm that my contribution statement and AI usage log accurately describe my work and assistance received, and that my reflection is my own writing. I can explain, test and modify the work claimed under my name. I have reviewed the evidence linked in this section.

Registered name: [student completes]. Student ID: [student completes]. Signature: [student signs after review]. Date: [actual signing date].

Student 4: Anoja

Primary component: D — Quality Inspection and NCR. Student ID and registered name require confirmation.

Contribution statement and technical work

Component responsibility proposed by the existing project ownership map: implement and integrate the API, relational entities, React screens, Flutter screens, tests and domain agent below. Student confirmation:

[write the specific work you personally implemented, reviewed and tested; identify shared work and collaborators]. Repository presence alone does not establish who wrote the code.

Inspection quantity reconciliation, automatic NCR creation and controlled resolution. Zero rejection creates no NCR; positive rejection creates one. Failed checklist items require notes, rejected quantities require reasons, and evidence count/size limits apply.

Database scope: Inspection, InspectionItem, InspectionEvidence, NonConformance. Inspect corresponding EF configurations and migrations for primary/foreign keys, relationships, constraints and audit fields. Explain how the component consumes upstream IDs and exposes downstream records without duplicating another component's source of truth.

Layer	Evidence file
ASP.NET Core API	backend/BuildWise.Api/Controllers/QualityInspectionsController.cs
Backend regression tests	backend/BuildWise.Api.Tests/QualityInspectionServiceTests.cs
React UI	web/buildwise-web/src/pages/QualityInspectionsPage.jsx
Flutter UI	mobile/buildwise_mobile/lib/features/operations/screens/quality_inspection_screen.dart
Flutter tests	mobile/buildwise_mobile/test/quality_inspection_test.dart
Distinct domain agent	backend/agent_service/quality_agent.py
Agent tests	backend/agent_service/test_quality_agent.py

Meaningful API endpoints: POST /api/quality-inspections; GET /api/quality-inspections; GET /api/quality-inspections/{id}; POST /api/quality-inspections/{id}/risk-analysis; GET /api/quality-inspections/non-conformances; POST /api/quality-inspections/non-conformances/{id}/transition. Route parameters are symbolic; inspect controller attributes for exact constraints. This supplies at least four component operations; explain authorization, input DTOs, success responses and invalid-input handling in the viva.

Distinct Agentic AI contribution

The component agent is `quality_agent.py` with the companion `test_quality_agent.py` suite. Document its input contract, permitted context/tool use, structured output and deterministic checks before a human consumes its recommendation. A model-generated rationale is advisory; the API retains business-rule enforcement. Student confirmation: [identify your personally implemented agent function, orchestration/state changes, rejected alternatives and verification]. Explain how a unavailable provider or malformed recommendation is surfaced rather than silently recorded as a valid decision.

Commit, pull-request and test evidence

The following is real path-based Git history, including the recorded author. It must not be relabelled as the named student's authorship. Add actual reviewed PR URLs or state that no PR evidence is available.

```
4e09d1c | Loganathan Theebika | Align React and Flutter workflows and refresh SE3090 submission documentation
c318c46 | Loganathan Theebika | Complete field mobile workspaces: approvals, review, PO, quotation entry, RBAC, validation UX
04d35a6 | Loganathan Theebika | Complete field mobile workspaces
9ab414f | IT24102414 | feat: RBAC hardening, supplier portal, budget validation and C1-C4 coverage
```

Commit URLs use <https://github.com/IT24102414/build/commit/> followed by the recorded hash. Personal commit/PR attribution: [student verifies]. Tests above belong to the observed group suites; component-specific passed counts must come from a recorded focused run, not a guessed division of the group totals.

Challenges and learning — student input

Describe one actual cross-stack contract problem, one invalid input or role boundary you tested, and one agent output you corrected or rejected. Reference the exact change/test. Explain what you can now implement or debug without external AI assistance. Do not claim these experiences until you have personally reviewed and reproduced them.

Individual AI usage log

The existing log `IT24XXXX-Anoja-ai-usage-log.md` is reproduced in the individual-log appendix under this student's label. Verify each date, tool/model, prompt/task, output, correction and test result; remove unsupported entries. Add the actual 5 October maintenance work you reviewed, distinguishing coding-assistant use from the application's own agents. Do not invent past prompts or dates.

Approximately one-page personal AI reflection — student authored

[Write approximately one page in your own words. Paragraph 1: tools used and stages of work. Paragraph 2: a useful output and a wrong output, tied to real evidence. Paragraph 3: what you changed, added or rejected and why. Paragraph 4: what you learned about your skills and limits, and how you verified ownership. Use your actual log and Git history. This report deliberately does not generate a first-person reflection for submission.]

Signed individual declaration — unsigned template

I confirm that my contribution statement and AI usage log accurately describe my work and assistance received, and that my reflection is my own writing. I can explain, test and modify the work claimed under my name. I have reviewed the evidence linked in this section.

Registered name: [student completes]. Student ID: [student completes]. Signature: [student signs after review]. Date: [actual signing date].

Part C: Submission links and references

Repository: <https://github.com/IT24102414/build>

React deployment: NOT PROVIDED. API/health/Swagger deployment: NOT PROVIDED. PostgreSQL deployment evidence: NOT PROVIDED. Current APK:

<https://github.com/IT24102414/build/raw/refs/heads/main/BuildWise.apk> (source commit 3b94c1e, artifact commit 23d5f22; release-mode build succeeded). Physical-device installation evidence: NOT PROVIDED. API endpoint embedded in this demo APK: <http://10.0.2.2:5078/api>, suitable for the Android emulator only. SHA-256: 7917092FC8B3C378C12920D4202AC4F04527630349ADE8453DE11ACDF055EBAD. Existing development-key signing is used; this is not a production-store signing assertion. Ten-minute video: NOT PROVIDED.

Primary requirements source: user-supplied SE3090 Assignment 1 specification, Year 3 Semester 1, 2026. Implementation references: root README, current source, EF migrations, test suites, docs/submission-readiness.md and the listed ADRs. Library dependencies and font licensing are recorded in package

manifests and assets/fonts/OFL.txt. This draft supplies no invented signatures, contribution history, performance figures or deployment results.

Observed local performance (2026-10-05)

30 requests per scenario, concurrency 5. All scenarios returned 100% successful responses.

Scenario	p95 ms	Target ms	Outcome
Shared ASP.NET Core API + PostgreSQL	78.89	2000	Passed
Agent 8001 quotation analysis	6782.05	1000	Latency target missed
Agent 8002 request analysis	94.49	1000	Passed
Agent 8003 discrepancy analysis	82.03	1000	Passed
Agent 8004 quality-risk analysis	58.02	1000	Passed

Quotation analysis missed the 1000 ms local target; investigate provider/rationale latency before claiming performance acceptance. No threshold was relaxed. API/database latency is measured together, not as an isolated database benchmark.

Submission completion and installation guide

This guide is embedded in the consolidated PDF. Submit that one PDF for written work. Deployment, personal authorship and signing fields remain unverified until the team supplies evidence.

Group declaration review

The one consolidated group declaration template appears in Part A, section 16. All four students must review it, confirm registered names and IDs, and sign with actual dates. Do not sign for another member or present the template as a completed declaration.

Environment variables and startup

Service	Names	Purpose
Local PostgreSQL scripts	POSTGRES_HOST, POSTGRES_PORT, POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD	Local database configuration; check .env.example
ASP.NET Core	ConnectionStrings__DefaultConnection	PostgreSQL connection secret
JWT	Jwt__Key, Jwt__Issuer, Jwt__Audience	Signing secret and token identity
Internal agents	AgentService__Url, AgentService__RequestUrl, AgentService__DeliveryUrl, AgentService__QualityUrl	Internal service origins corresponding to ports 8001–8004
SMTP	Smtp__Host, Smtp__Port, Smtp__EnableSsl, Smtp__Username, Smtp__Password, Smtp__FromAddress, Smtp__FromName	Real provider configuration; an empty host does not deliver mail
Optional model providers	GEMINI_API_KEY, ANTHROPIC_API_KEY, ANTHROPIC_MODEL, ANTHROPIC_TIMEOUT_S	Provider configuration used by agent source
React build	VITE_API_BASE_URL	API origin ending in /api
Flutter build	API_BASE_URL via --dart-define	API origin embedded at build time

Never include secret values in the report, screenshots, video or Git. Keep PostgreSQL and agents private; clients call the API. Configure deployed CORS against the actual React origin.

1. Install prerequisites from README and create a private .env from .env.example. Configure PostgreSQL and JWT using environment variables or dotnet user-secrets.
2. In backend/agent_service create a Python environment and install requirements.txt. Run npm ci in web/buildwise-web and flutter pub get in mobile/buildwise_mobile.
3. Run dotnet ef database update --project backend/BuildWise.Api after configuring the database connection.
4. From the repository root run powershell.exe -NoProfile -ExecutionPolicy Bypass -File scripts/start-dev.ps1 -TimeoutSeconds 30.
5. Verify local React at <http://127.0.0.1:5173>, health at <http://127.0.0.1:5078/health>, Swagger at <http://127.0.0.1:5078/swagger> and agent services. Local API origin is <http://127.0.0.1:5078/api>. Start Flutter with the appropriate device URL.

Detailed isolated startup commands: docs/component2_setup_guide.md. SMTP guidance: docs/email-setup.md. Seeded demo password is Passw0rd!; demo accounts are for evaluation, not production.

APK installation and verification

Download: <https://github.com/IT24102414/build/raw/refs/heads/main/BuildWise.apk>. Fresh 5 October release-mode build from source 3b94c1e, published in commit 23d5f22. SHA-256: 7917092FC8B3C378C12920D4202AC4F04527630349ADE8453DE11ACDF055EBAD. Uses the existing development signing key; this is not a production-store signing assertion.

The supplied APK embeds <http://10.0.2.2:5078/api> for an Android emulator on the API host computer. Download, permit installation from that source if Android asks, install, start the API and agents, and sign in. Alternatively use `adb install -r BuildWise.apk`. A conflicting older signing key may require removing an old installation; preserve needed data first.

For a physical phone, obtain a reachable public HTTPS API or LAN API host and rebuild:

```
cd mobile/buildwise_mobile

flutter build apk --release --dart-define=API_BASE_URL=https://YOUR_API_HOST/api
```

YOUR_API_HOST is a placeholder, not a deployed address. Verify login, seven roles, request/approval/receiving/inspection flow, camera/gallery permissions, evidence display, notifications, invalid inputs and logout on the installed device. Record device/Android version, checksum, date and real outcomes. Widget tests do not prove physical-device behavior.

Manual evidence still required

Required item	Status	What the team must supply
Public React URL	Not supplied	Reachable app and seeded role verification
Public API/health and Swagger URLs	Not supplied	Accessible endpoints and sanitized hosting evidence
PostgreSQL deployment	Not supplied	Provider/database identity, migration success and sanitized connectivity evidence
Agent access/setup	Local setup available	Deployment configuration and actual persisted workflow execution
Four student identities	Incomplete	Confirm registered names and IDs
Contribution/PR evidence	Not personally verified	Actual individual attribution and reviewed PR URLs, or state that none exist
Individual reflections and logs	Student review required	Personally written reflections and corrected chronological logs
Individual/group signatures	Not supplied	Review and actual signatures/dates
Device APK execution	Not supplied	Installed-device evidence using reachable API
Ten-minute video	Not supplied	Accessible link and actual demonstrated lifecycle

Keep evaluator access until at least 21 October 2026. The specification deadline was 30 September 2026 at 11:50 PM; this 5 October revision does not certify an on-time submission.

Test and agent evaluation review

Observed group evidence: 307 backend, 28 Python, 163 React and 149 Flutter tests passed. Later styling verification passed 13 navigation/material-request tests, and the added administrator email-correction regression passed separately. No new aggregate Flutter count is inferred. Flutter analysis was clean and the release APK build succeeded.

Explain negative tests for invalid quantities, unauthorized roles, quotation constraints, NCR transitions, malformed agent recommendations, budget checks and email failures. Record new manual runs with input, expected result, actual result, pass/fail and artifact reference. Do not fill actual outputs before running a case.

Live local performance used 30 requests per scenario at concurrency 5. Quotation-agent p95 was 6782.05 ms against a 1000 ms target and missed it. Investigate provider/rationale latency, measure deterministic and provider-enabled modes separately, and rerun the same workload. Do not lower the threshold to disguise failure. API latency includes database work rather than an isolated SQL benchmark.

Agent evaluation must distinguish deterministic fixture tests from live provider quality. Record schema adherence, rejected invalid outputs, human decisions, permitted tools, persisted state and failure handling. No invented accuracy, baseline, token-cost or user-study result is included.

Final four-student review and references

Each student confirms their component, reproduces positive and negative tests, explains four API operations plus its domain operation, reviews database relationships and distinct agent, then writes their own contribution statement, AI chronology, one-page reflection and signed declaration. Shared commits remain shared evidence; do not rewrite one author's Git history as four independent histories.

The group checks diagrams against current migrations, verifies deployments and device APK, records the video, signs the group declaration, regenerates and inspects the PDF. Submit one consolidated PDF instead of separate individual report PDFs.

Primary reference: user-supplied SE3090 Assignment 1 specification, Integrated Full-Stack and Agentic AI Application Development, Year 3 Semester 1, 2026, 17 pages. Sections 3–9 cover component/full-stack/agent expectations; 14–16 cover reporting/submission/rubric; 18–19 cover AI disclosure and ownership. Repository paths and actual Git history are implementation references. Historical deployment ADR proposals are not proof of hosting.

Student 1: Peiris DPSS: historical AI log appendix

Historical student log reproduced for review. Personal authorship, chronology, reflections and signatures must be verified by the student before submission; reproduction is not certification.

AI Usage Log — Component 1: Material Request & Approval Management

Student: Peiris DPSS · **ID:** IT24XXXXX · **Date range:** 2026-09-10 – 2026-09-30

1. Declaration

I used AI tools (GitHub Copilot, Claude, and ChatGPT) to assist in the development of Component 1 (Material Request & Approval Management). All architectural and domain decisions were my own. All generated code, database configurations, and test assertions were reviewed, tested, and modified before committing to version control.

2. Session Log

#	Date	What was built / investigated	Issues found	How verified	Evidence
1	2026-09-10	Domain entity design (MaterialRequest, MaterialRequestItem, Approval, MaterialRequestHistory)	Copilot omitted composite FK constraints for multi-item requests.	Configured EF Core EntityTypeConfiguration manually.	MaterialRequest.cs, MaterialRequestItem.cs
2	2026-09-14	.NET 8 MaterialRequestsController CRUD and approval workflow	Status transition guard was missing on re-approval; allowed duplicate approvals.	Added state transition checks and unit tests.	MaterialRequestsController.cs
3	2026-09-18	RequestAnalysisAgent (Python FastAPI on Port 8002) for site urgency and quantity risk analysis	Agent initially returned unparsed text instead of typed JSON flags.	Implemented Pydantic schema validation for output structure.	request_agent.py, test_request_agent.py
4	2026-09-22	React MaterialRequestsPage.jsx with search, filter, and review drawer	Initial table rendered without responsive mobile wrapping or loading spinners.	Added loading states, status badge tones, and client-side filters.	MaterialRequestsPage.jsx, MaterialRequestsPage.test.jsx
5	2026-09-25	Flutter mobile screen (material_requests_screen.dart)	Form allowed negative quantities and lacked unit selection.	Added input validation and unit dropdown with discrete integer checks.	material_requests_screen.dart
6	2026-09-28	RBAC integration and approval security (Policies.MaterialRequestApprovalOnly)	SiteEngineer was initially able to trigger approval endpoint if token was forged.	Enforced ASP.NET Core policy returning 403 Forbidden.	RbacAuthorizationTests.cs
7	2026-09-30	Full lifecycle E2E testing & documentation	Material request history rows were not written on revision status.	Updated MaterialRequestService.cs to commit audit history.	MaterialRequestServiceTests.cs, FullLifecycleScenarioTests.cs

3. Reflection

What the AI tools did well

- **Scaffolding:** Rapidly generated initial DTO boilerplate and REST controller action templates in C#.
- **Test Generation:** Assisted in creating edge-case test vectors for quantity boundaries and invalid date submissions.
- **Python Fast-API:** Streamlined FastAPI schema setup for the Request Analysis Agent.

Where the AI tools needed correction

- **EF Core Mapping:** AI models frequently forgot navigation property includes (`.Include(mr => mr.Items)`), leading to null reference exceptions in query projections.
- **RBAC Security:** AI suggested putting role authorization inside controller bodies rather than using declarative `[Authorize(Policy = "...")]` attributes.
- **State Machine Guardrails:** AI did not implement business rules preventing modifications to already-approved requests.

Lessons Learned

1. AI output must always be verified against formal domain specifications.
2. Declarative policy-based security in ASP.NET Core is far superior to ad-hoc procedural role checks.
3. Strict schema validation (Pydantic / C# DTOs) is essential at every AI agent boundary.
4. Testing edge cases (negative quantities, past dates, duplicate submissions) catches subtle integration bugs early.
5. Cross-platform API integration requires consistent DTO contract versioning across React and Flutter.

What I would do differently

In future projects, I would establish contract-first OpenAPI schemas and generate client types automatically before implementing UI screens. I would also write integration tests earlier in the sprint.

4. Signed Declaration

I confirm that this log is an accurate record of my AI tool usage for SE3090 Assignment 1. I can explain, modify, and debug every line of code submitted under Component 1.

Student Name: Peiris DPSS

Student ID: IT24XXXXX

Date: 30 September 2026

Signature: *Peiris DPSS*

Student 2: Theebika: historical AI log appendix

Historical student log reproduced for review. Personal authorship, chronology, reflections and signatures must be verified by the student before submission; reproduction is not certification.

AI Usage Log — IT24102414

Component 2 — Supplier, Quotation & Procurement Management

BuildWise · SE3090 Software Engineering Frameworks · branch feature/supplier-procurement

- > **Note to author:** the **Chronology** section below is a factual record and can
- > be verified against git history and CI runs. The **Reflection** section is a
- > **scaffold** — the conclusions offered should be re-read, edited or rejected so
- > the final wording is genuinely yours.

1. AI tooling used

Tool	Role
Cline (agentic coding agent, Claude model)	Repo inspection, running test suites, driving live HTTP/DB verification, authoring fixes
Claude (via the agent's optional LLM rationale path)	Production rationale generation for the agent — not exercised in this run (ANTHROPIC_API_KEY unset, so llm_rationale_enabled: false)

2. Technical chronology

#	Event	Evidence
1	Initial scope acknowledgement request refused without evidence; repo located at Downloads/buildwise-component2	git log abdc6d5, author IT24102414
2	Baseline verified: 15 C# tests, 9→7 Python tests, 15 React tests, npm run build exit 0	test runs
3	Bug found (Python): a quotation marked <code>valid = false</code> produced an "excluded" warning but was <i>*not*</i> skipped — it was ranked and could win top spot. Demonstrated live against <code>POST /analyze</code>	quotation_a gent.py filter_elig ible
4	Bug found (Python): the resilient fallback re-admitted every quotation the filter had just rejected, with <code>covers_all</code> forced <code>True</code> — a fully Suspended/Inactive supplier could be recommended as "lowest-cost compliant ... Active standing"	analyze() lines ~196– 197
5	Both fixed in 38bbca9: continue on invalid quotations, fallback gated on <code>supplier_status == "Active"</code> and <code>valid</code> ; two regression tests added (7 → 9 Python tests)	commit 38bbca9
6	Doc/code mismatch found: checklist expected <code>{"status": "healthy", "service": "quotation_agent"}</code> but the endpoint returned <code>{"status": "ok", "agent": "..."}.</code> Verified nothing in the repo calls <code>/health</code> , so the doc was wrong, not the code	grep of .cs/.js/.js x, .github/
7	<code>/health</code> updated to the documented contract as a superset; the existing <code>test_health_endpoint</code> assertion updated alongside it (it asserted "ok" and would otherwise have failed pytest)	commit 38bbca9
8	Bug found (C#↔Python integration): every <code>POST /procurement-workflow</code> failed with "recommended_quotation_id is missing or non-positive" and 0 of 50 workflows ever succeeded, 0 purchase orders ever created. Root cause: <code>AgentRecommendationDto</code> is <code>PascalCase</code> with no <code>JsonPropertyName</code> , while the agent emits <code>snake_case</code> — so the HTTP 200 response deserialized to an all-null object. Proven by the success log line "Agent analysis received from external microservice" plus the absence of any "unreachable" warning	agent_workf low_steps = 0 stored results; purchase_or ders count 0

9	Fixed in 6161ecc: response read with <code>JsonNamingPolicy.SnakeCaseLower</code> , scoped to that read only (React camelCase contract preserved)	commit 6161ecc
10	Full E2E replay after the fix: workflow #51 → <code>AwaitingApproval</code> → <code>Approved</code> → purchase order created (250 bags @ 2 100.00 = 525 000.00); quotations #3 Selected, #6/#7 Rejected	DB rows + API responses
11	Anomaly left open: the PO-creation call that succeeded server-side returned HTTP 400 to the client. Not reproducible — the identical request returned 200 OK. Suspected EF Core tracking conflict when <code>CreateFromWorkflow</code> re-enters <code>GetById</code> on the same scoped <code>DbContext</code> . Recorded in docs/adr/ADR-002 §6	logs + reproduction attempt
12	Test baseline after fixes: 15 C#, 9 Python, 15 React, build exit 0	test runs

2b. Addendum — changes observed in the working tree after this session

The following edits appeared in the working tree afterwards and were ****not authored in this session****. They are recorded because they bear directly on the findings above:

- `Program.cs` now defaults `AgentService:Url` to port **8001** (was **8000**), closing the "silently skip the agent and use the fallback" risk described in §5 of docs/adr/ADR-002.
- `ProcurementWorkflowService.GetWorkflowDetailsAsync` now reads the recommendation only from the analysis agent's step (`AgentRole == QuotationSupplierAnalysisAgent`), so the planning step's plan JSON can no longer be mistaken for a recommendation payload.
- `backend/BuildWise.Api.Tests/AgentRecommendationSchemaTests.cs` adds the boundary test that was missing: a stubbed `HttpMessageHandler` returns a `snake_case` agent payload and asserts it survives the real deserialization path, plus §5.7 schema-gate and §10 failure-handling cases. The C# suite is now **20** tests (was 15).
- `scripts/verify-component2.ps1` (with `check-services.ps1`, `smoke-test.ps1`, `start-dev.ps1`, `stop-dev.ps1`) provides one-command verification of the whole component, including the approval → purchase-order path.

3. What the AI tooling did well

- **Found both bugs by execution, not by reading.** Neither the "invalid quotation could win" defect nor the deserialization failure was discoverable from the unit tests, which were green throughout. Both were surfaced by driving the real HTTP endpoint with a deliberately adversarial payload

(cheapest supplier suspended, cheapest alternative partial, one quote expired) and by comparing the agent's raw answer with what the API had stored.

- **Proved causation instead of guessing.** The deserialization diagnosis was established from four independent facts: the Python answer was correct, the stored recommendation was all-null, the "success" log line was present, and no "unreachable" warning was. The conclusion followed from evidence, not intuition.

- **Checked side effects.** After `npm run build` rewrote `dist/`, it confirmed the output hashes were byte-identical to what was committed and that the tree stayed clean.

- **Resisted an over-broad change.** When the `/health` payload mismatched the checklist, it first grepped the codebase for consumers before concluding the documentation was the wrong side — and it also caught that Step 2 of the intended fix would have broken an existing test.

4. Where the AI tooling needed human correction

- **A strict "do not run tools" instruction had to be refused.** Acknowledging "Component 2 is 100% complete" would have asserted passing tests and pushed commits that had not been checked.

- **Scope discipline needed.** It initially wanted to patch the Python bugs unprompted; the branch was already pushed and graded, so the edits were deferred until explicitly approved.

- **Two claims had to be walked back.** `dotnet test` without a project path failed with MSB1003 (no solution file at the repo root) and produced a silently empty result; and the HTTP 400 root cause was reported before a reproduction attempt existed.

- **Unrequested state changes.** The verification mutated the local dev database (supplier status changed, quotations deleted/added, a PO created). This was disclosed, but it is a real hazard of letting an agent run against a database.

- **Partial verification only.** The React UI was never clicked in a browser; the "end-to-end" run was API-level. Visual rendering was not verified.

5. Lessons

1. **Green tests do not mean a working system.** All 37 tests passed while the two most important features (expired-quote exclusion, agent→API integration)

were broken. The tests that mattered were integration tests that did not exist yet.

2. **Cross-language boundaries are where contracts break.** The request DTO was deliberately snake_case to match the agent, and the response DTO was PascalCase with no mapping — the asymmetry is invisible until you look at the wire format. A schema test on the actual HTTP boundary would have caught it; one has since been added (AgentRecommendationSchemaTests.cs), stubbing the agent's HTTP handler so the boundary is tested without the network.

3. **Defense in depth is what made the failure survivable.** The C# validation agent rejected every bad recommendation, so the integration bug produced "workflow failed" instead of a wrong purchase order.

4. **Degrade loudly, not silently.** A dead agent silently falls back rather than erroring — resilient, but it also hides an outage behind a working system.

5. **An agent must not write data it is not accountable for** — hence the approval gate and the read-only exposure of purchase_orders to Component 3.

6. Reflection (draft — personalise before submission)

<!--

DRAFT SCAFFOLD. The claims above in sections 2–5 are factual and verifiable; the wording below is offered as a starting point. Edit it so it reads as your own account of your own decisions, and delete this comment.

-->

Working with an AI coding agent on Component 2 changed what I could verify, not just how fast I wrote code. The most valuable thing it did was **not** writing code — it was refusing to accept "everything is done" on my say-so. When I asked it to confirm the component was complete, it declined and asked to verify first, and that verification is what found a bug in the AI agent itself: a quotation marked invalid was being warned about as "excluded" while still winning the recommendation. That is exactly the failure the spec (§5.3) is supposed to prevent, and all 37 existing tests were green while it happened.

The second finding was more instructive. Every one of the 50 procurement workflows had failed, and no purchase order had ever been created, even though all the tests passed. The cause was a mismatch between how the C# API named its fields and how the Python agent named its — the request direction had been

carefully aligned and the response direction had not. What I take from that is that an integration between two languages needs a test on the boundary itself, not just on either side of it. The fix was three lines, but finding it took running the real system, not reading the real system.

There are things I would do differently. I let the agent change a development database during verification, which was necessary to reproduce the scenario but should have been sandboxed first. I also accepted an "HTTP 400 on success" as explained for a while before admitting I had not actually reproduced it, and I eventually recorded it as an open question in the ADR instead of guessing a cause. That discipline — separating what I proved from what I suspect — is the part of this exercise I expect to keep using.

Student 3: Ramya: historical AI log appendix

Historical student log reproduced for review. Personal authorship, chronology, reflections and signatures must be verified by the student before submission; reproduction is not certification.

AI Usage Log — Component 3: Delivery & Material Receiving Management

Student: Ramya · **ID:** IT24102513

Branch: feature/Delivery-And-Material-Receiving-Ramya-IT24102513

Merged into: integration/component2-plus-c3

Date range: 2026-09-10 – 2026-09-27

1. Overview

This log documents every session in which GitHub Copilot or Claude (via the Windsurf IDE chat) was used during the development of Component 3. The component covers the full delivery and material receiving lifecycle: Scheduled → InTransit → Delivered / PartialDelivery / Rejected, the DeliveryDiscrepancyAgent Python microservice (port 8003), the RequestAgent microservice (port 8002), the ASP.NET Core controller and service layer, and the React front-end page with its CSS.

2. Session Log

#	Date	What was built / attempted	Bugs / issues encountered	How verified	Evidence
1	2026-09-10	Scaffolded entity models (Delivery.cs, DeliveryItem.cs, DeliveryIssue.cs, DeliveryEvidence.cs, DeliverySchedule.cs) and the initial EF Core migration. Copilot generated the class skeletons and navigation properties from a prompt describing the delivery lifecycle.	DeliverySchedule had a foreign key to purchase_orders (Component 2's table). When the migration was applied on top of C2's schema, the FK constraint failed — the purchase_orders table did not yet exist in the shared integration branch because C2's migration had not been ordered first. Copilot had no awareness of the cross-component migration ordering requirement.	Re-ordered the migrations manually; ran dotnet ef database update against the local integration database and confirmed the FK resolved.	Migration file timestamp corrected; schema applied cleanly on integration/component2-plus-c3.
2	2026-09-13	Built DeliveryService.cs — core receiving logic: status resolution (Received, PartialDelivery, Rejected), PO line-item matching, DeliveryIssue creation on discrepancy. Copilot drafted the status-resolution switch and the LINQ query for matching delivery items against PO items.	After initial generation, partial deliveries were recorded with status PartialDelivery at the delivery level, but the corresponding purchase order line items were not updated to reflect the partial fulfilment. This meant Component 4 (quality inspection trigger) could not determine how much stock had actually arrived. Copilot's suggestion silently omitted the PO line-item write-back.	Added a unit test in DeliveryServiceTests.cs that asserted PurchaseOrderItem.FulfilledQuantity after a partial receive. Test failed on first run; fix was written manually.	DeliveryServiceTests.cs — test RecordDelivery_PartialReceive_UpdatesPurchaseOrderLineItems now passes as part of the 230/230 total.

3	2 0 2 6- 0 9- 1 5	Built DeliveryAgentService.cs, the C# client that calls the Python DeliveryDiscrepancyAgent. Claude generated the HttpClient call, JSON serialisation, and the response-mapping DTO.	The generated code targeted port 8001 (the Quotation Agent from Component 1) instead of 8003. The agent also returned a flat { shortage_detected, damage_detected, summary } structure initially, but the C# service expected a richer { discrepancies, summary } schema with separate shortageQuantity and damagedQuantity fields. Two separate corrections were needed: fix the port constant, then align the Python response schema.	Ran test_delivery_agent.py (3 tests: test_analyze_discrepancy_shortage, test_analyze_discrepancy_damage, test_analyze_discrepancy_none) after each fix. All 3 passed once port and schema were corrected. Integration smoke test: DeliveryAgentServiceTests.cs confirmed the C# client mapped the response correctly.	delivery_agent.py line 43: uvicorn.run(app, host="127.0.0.1", port=8003). DeliveryAgentServiceTests.cs contributes to the 230/230 .NET total; test_delivery_agent.py contributes to the 28/28 Python total.
4	2 0 2 6- 0 9- 1 7	Built request_agent.py (port 8002) — the RequestAgent that flags material requests as HIGH_URGENCY, BULK_ORDER, or LARGE_QUANTITY_ORDER based on reason text and quantities. Claude generated the FastAPI route and flag logic from a natural-language description of the rules.	The initial generated endpoint accepted items_count but had no total_quantity field, so LARGE_QUANTITY_ORDER could never be flagged. Claude omitted the field because I described the rules verbally without explicitly listing all parameters. Adding total_quantity: float = 0 to RequestAnalysisInput fixed it.	test_request_agent.py — 3 tests cover HIGH_URGENCY, BULK_ORDER, and a normal (empty flags) request. A fourth test for LARGE_QUANTITY_ORDER was written manually and confirmed the field was now present. The 4 tests contribute to the 28/28 Python total.	request_agent.py line 19: total_quantity: float = 0; test_request_agent.py passes in CI.
5	2 0 2 6- 0 9- 1 9	Built DeliveriesPage.jsx — the React page with DeliveryList, ReceiveDeliveryForm, and DeliveryDetail sub-components. Also wired the qualityApi.analyzeDeliveryDiscrepancy call into the detail panel. Copilot generated the initial component tree.	(a) The page had no CSS at all — plain HTML with no layout classes. Copilot did not generate a stylesheet unprompted. (b) The generated deliverySupplier() helper read delivery.purchaseOrder.supplier.name — a nested structure the DTO never contained. Every row displayed "Supplier record not expanded" even though the API was sending a flat supplierName field.	(a) DeliveriesPage.css written separately (41 lines, responsive grid for metric cards, receiving lines, drawer layout). (b) Fixed deliverySupplier() to read delivery.supplierName first (line 184 of DeliveriesPage.jsx). Regression test it('renders the supplier and material names returned by the API') was added to DeliveriesPage.test.jsx and confirmed fix.	DeliveriesPage.css (41 lines, 3 590 bytes); DeliveriesPage.test.jsx line 103–115 covers supplier name regression.
6	2 0 2 6- 0 9- 2 2	Wrote DeliveriesPage.test.jsx — 18 tests across three describe blocks: main page (DeliveriesPage), record form (DeliveriesPage (record form is optional)), and detail drawer (DeliveriesPage (detail drawer)). Claude suggested the vi.mock patterns and userEvent flow; test bodies were written collaboratively.	Test fixtures initially used the nested purchaseOrder.supplier.name shape (matching the broken production code), so tests passed while the page was wrong. After fixing both the component and the fixtures to use supplierName, the regression test correctly caught re-introduced bugs. Also: the it('caps the received input at the ordered quantity') test initially asserted max="100" as a number, but the DOM attribute is a string; Claude's assertion was wrong and needed toHaveAttribute('max', '100').	All 18 tests pass in Vitest. The suite covers: loading state, error state, empty state, filtering, supplier/material name rendering, drawer open/close, Escape key, role-based access, quantity cap, form submission, agent run, agent failure graceful degradation, stale-analysis clearing.	DeliveriesPage.test.jsx — 364 lines, 18 it() blocks.

7	2 0 2 6- 0 9- 2 4	Built DeliveriesController.cs (11 468 bytes) — 8 endpoints: list, get, record, update status, analyse discrepancy, list issues, list evidence, list schedules. Copilot generated the controller shell and route attributes from a summary of the domain entities.	The [HttpPost("{id}/analyze-discrepancy")] action initially called <code>_deliveryAgentService.AnalyzeAsync(id)</code> but forgot to await the async result and returned a <code>Task<ActionResult></code> object rather than the analysis DTO. Copilot generated <code>return Ok(result)</code> on the same line as <code>var result = _deliveryAgentService.AnalyzeAsync(id)</code> without <code>await</code> . This compiled without warning because the return type was already <code>Task<IAActionResult></code> .	Manual code review caught the missing <code>await</code> . <code>DeliveryServiceTests.cs</code> integration test for the <code>analyze</code> path confirmed the fix.	DeliveriesController.cs — 11 468 bytes; review comment left in the file on the corrected line.
8	2 0 2 6- 0 9- 2 7	Final integration pass: confirmed branch merged into <code>integration/component-2-plus-c3</code> , all 230/230 .NET tests green, all 28/28 Python tests green, <code>DeliveriesPage.test.jsx</code> 18/18 Vitest tests green. Reviewed inter-component contracts: delivery references a <code>purchase_orders</code> PK (Component 2), delivery completion triggers quality inspection event (Component 4).	Minor: the <code>DeliveryDetail</code> component's <code>useEffect</code> reset <code>analysis</code> and <code>analysisError</code> on <code>delivery?.id</code> change, but the dependency array initially listed <code>delivery</code> (the whole object) rather than <code>delivery?.id</code> . This caused the effect to fire on every parent re-render that produced a new object reference, clearing an in-progress analysis mid-flight.	Changed dependency to <code>delivery?.id</code> (line 306 of <code>DeliveriesPage.jsx</code>). <code>Test it('clears a previous analysis when a different delivery is opened')</code> (line 348) confirmed the reset fires at the right moment.	<code>DeliveriesPage.jsx</code> line 304–307; test at line 348 passes.

3. Reflection

3.1 What AI did well

GitHub Copilot was genuinely useful for generating repetitive but necessary code: entity property lists, navigation-property declarations, LINQ expressions for status resolution, and FastAPI Pydantic models. The boilerplate saved time that was better spent on domain logic and cross-component integration. Claude was strong at producing `vi.mock` patterns and `userEvent` interaction sequences for the React test suite — patterns that would have taken longer to look up from scratch. For the Python microservices, Claude's first-pass FastAPI route structures (`DiscrepancyInput`, `RequestAnalysisInput`, health endpoints, `uvicorn` startup blocks) were accurate and consistent across both `delivery_agent.py` and `request_agent.py`.

3.2 Where AI failed or needed correction

Three classes of failure appeared consistently.

Cross-component awareness was zero. The FK ordering bug in the `DeliverySchedule` migration (Session 1) and the `port-8001` mistake in `DeliveryAgentService` (Session 3) both happened because the AI had no knowledge of the broader team's architecture. Copilot defaulted to whatever port or table name it had seen most recently in context, which happened to be wrong in both cases. These are the kind of mistakes that slip through if the developer does not actively review generated code against the integration spec.

DTO shape assumptions were wrong. The `deliverySupplier()` helper (Session 5) and the initial Python response schema (Session 3) both assumed a more deeply nested data model than the actual API produced. AI generated code that was internally consistent but did not match the real wire format. The supplier name bug was particularly harmful because all existing tests passed — they were also using the wrong fixture shape. The lesson is that AI-generated test fixtures must be validated against real API responses, not just against the component under test.

Async correctness was missed. The missing `await` in `DeliveriesController.cs` (Session 7) compiled cleanly and would have silently returned a task object wrapped in HTTP 200. This is a category of error where the AI produces syntactically valid but semantically broken code, and only careful human review catches it.

3.3 Lessons learned

1. Always cross-reference AI-generated port numbers, table names, and endpoint paths against the integration specification before running tests. The AI only knows what is in its context window.
2. Write at least one test that uses real API fixture shapes — copied from a real response or from a shared DTO — before accepting AI-generated component code. Fixing the `deliverySupplier()` bug required fixing both the component *and* the test fixtures simultaneously.
3. Async C# code generated by AI should be reviewed line-by-line for missing `await` keywords, especially in controller actions where the return type already wraps `Task<IActionResult>` and the compiler has no way to warn.
4. AI will not volunteer a CSS file. If the visual output matters, it must be asked for explicitly and reviewed separately.

3.4 What I would do differently

I would define the inter-component contracts (port numbers, table names, DTO field names, migration ordering) in a shared reference document at the start of the sprint and paste the relevant section into every AI prompt. This would have prevented the port-8001 mistake and the FK ordering failure with minimal extra effort. I would also write at least one "real shape" fixture test (using data copied from a live or seeded API response) before writing any AI-assisted component logic, so that DTO shape mismatches surface immediately rather than after several sessions of passing tests.

4. Signed Declaration

I declare that this log accurately and honestly reflects my use of AI tools during the development of Component 3 of the BuildWise system. All AI-generated code was reviewed, tested, and corrected by me before being committed to the branch. The bugs documented above are real issues encountered and resolved during development. AI was used as an aid; all design decisions, domain logic, integration choices, and final code ownership remain my own.

Student: Ramya

ID: IT24102513

Component: Component 3 — Delivery & Material Receiving Management

Date: 2026-09-30

Student 4: Anoja: historical AI log appendix

Historical student log reproduced for review. Personal authorship, chronology, reflections and signatures must be verified by the student before submission; reproduction is not certification.

AI Usage Log — Component 4: Quality Inspection & Non-Conformance Management

Student: Anoja · **ID:** IT24XXXXX · **Component:** 4 — Quality Inspection & Non-Conformance Management
Branch: feature/quality-inspection · **Date range:** 2026-09-15 – 2026-09-29

1. Overview

This log documents every AI-assisted session during the development of Component 4 of the BuildWise platform. The component covers the full quality assurance lifecycle: recording inspection outcomes against received deliveries, automatically raising Non-Conformance Reports (NCRs) for rejected material lines, and managing NCR resolution through a role-gated workflow. The most distinctive artefact is `quality_agent.py` — the largest Python microservice in the project at 7,286 bytes — which implements the `QualityRiskAnalysisAgent` and exposes two analytical endpoints on port 8004.

2. Session Log

#	Date	What was built / worked on	Bugs or issues found	How verified	Evidence artefacts
1	2026-09-15	Scaffolded the four entity models — <code>Inspection.cs</code> , <code>InspectionItem.cs</code> , <code>InspectionEvidence.cs</code> , <code>NonConformance.cs</code> — and four supporting enums (<code>InspectionDecision</code> , <code>InspectionStatus</code> , <code>NonConformanceSeverity</code> , <code>NonConformanceStatus</code>). Used Copilot to generate the initial EF Core [Required] annotations and navigation properties.	Bug 1 (FK orphan rows): Copilot's first draft of <code>InspectionItemConfiguration.cs</code> omitted the explicit <code>.HasForeignKey(x => x.InspectionId)</code> in the Fluent API configuration. EF Core therefore created the <code>InspectionItems</code> table without a proper FK constraint. The migration ran without error but allowed <code>InspectionItem</code> rows to exist without a parent <code>Inspection</code> , creating orphaned rows in test data.	Detected during <code>dotnet ef migrations add review</code> — the generated SQL had no <code>FOREIGN KEY ... REFERENCES Inspections(Id)</code> clause. Fixed manually by adding <code>.WithMany(i => i.Items).HasForeignKey(x => x.InspectionId).OnDelete(DeleteBehavior.Cascade)</code> to <code>InspectionItemConfiguration.cs</code> , then re-running the migration.	EF Core migration file diff; <code>InspectionItemConfiguration.cs</code>

2	2 0 2 6 - 0 9 - 1 7	Built QualityInspectionsController.cs (7,662 bytes). Asked Claude to draft the POST /api/quality-inspections endpoint, which needed to: (a) validate the five-point checklist booleans, (b) persist InspectionEvidence records, (c) derive overallDecision from the checklist state, and (d) auto-raise NonConformance rows for any InspectionItem with rejectedQuantity > 0.	Claude's first draft computed overallDecision as "Pass" or "Fail" (strings), which did not match the InspectionDecision enum values (Accepted, PartiallyAccepted, Rejected). The enum mismatch caused a serialisation error at runtime. Separately, Claude omitted the supplierId link in the auto-raised NonConformance; the NCR was saved with SupplierId = 0, which broke the verify script's assertion at line 67 (Assert-True ([int]\$ncr.supplierId -eq [int]\$delivery.purchaseOrder.supplierId)).	Both issues caught during first run of verify-component4.ps1. Fixed by replacing Claude's string literals with the correct enum, and by reading delivery.PurchaseOrder.SupplierId in the NCR auto-raise logic inside the controller.	QualityInspectionsController.cs; verify-component4.ps1 check 3 (NCR supplier link)
3	2 0 2 6 - 0 9 - 1 9	Developed QualityInspectionService.cs and the NCR status-transition endpoint (POST /api/quality-inspections/non-conformances/{id}/transition). Asked Gemini to draft the allowed-transition guard and resolution-required logic.	Bug 3 (unguarded NCR transitions): Gemini's draft had no role check on the transition endpoint — any authenticated user, including a QualityInspector, could move a Critical NCR to Closed without providing a resolution text. The service layer also lacked a guard requiring resolution to be non-empty for the Resolved, Closed, and AcceptedException statuses.	Discovered via manual testing: logging in as quality.inspector@buildwise.de mo and posting { "status": "Closed" } to the transition endpoint succeeded. Fixed by (a) adding a [Authorize(Roles = "ProcurementManager, SiteManager, Administrator")] attribute to the controller action and (b) adding server-side validation that rejects transitions to resolution-bearing statuses if resolution is null or whitespace. The verify script (lines 75–80) confirms the manager-only path.	QualityInspectionService.cs; verify-component4.ps1 checks 4–5 (NCR transition); NonConformancesPage.jsx client-side mirror at line 76

4	2 0 2 6 - 0 9 - 2 1	Created <code>quality_agent.py</code> (7,286 bytes, port 8004). This is the most complex Python agent in the project. Copilot generated the two FastAPI endpoints — <code>/api/agent/analyze-quality-risk</code> and <code>/api/agent/recommend-ncr</code> — along with the Pydantic models <code>QualityRiskInput</code> , <code>QualityRiskOutput</code> , and <code>NcrRecommendationOutput</code> .	Bug 1 (type mismatch — string vs. numeric severity): The very first Copilot draft of <code>NcrRecommendationOutput</code> typed <code>ncr_severity</code> as a numeric score (0.0–1.0) in the docstring comment, but the actual returned value in the response body was a plain string label ("high", "medium"). This lowercase string did not match the <code>NonConformanceSeverity</code> enum values (Critical, High, Medium, Low) expected by the C# consumer. The failure was silent: the C# <code>JsonSerializer</code> silently mapped the unrecognised string to the enum default rather than throwing. Caught by reading the integration response manually. Fixed by changing the field to return PascalCase strings matching the C# enum (e.g., "Critical", "High") and adding an explicit <code>Pydantic Literal</code> constraint. Bug 6 (prompt injection surface): Copilot's original draft passed <code>item.rejection_reason</code> verbatim into the LLM system prompt. In this agent supplier names from the NCR description would have gone unescaped into the prompt context, creating a prompt injection vector. The agent was refactored to compute the risk level and corrective action through pure deterministic logic (the <code>if total_rejected > 50 / > 20 / > 0</code> branching now visible in lines 92–124) rather than via any LLM call, eliminating the injection surface entirely.	Unit tests in <code>test_quality_agent.py</code> (28 tests total across all agents; quality agent contributes the 12 tests in <code>TestQualityRiskAnalysis</code> and <code>TestNcrRecommendation</code> classes). Type mismatch confirmed by integration test comparing C# deserialised <code>NcrSeverity</code> enum value against expected.	<code>quality_agent.py</code> lines 19–50, 92–134, 163–192; <code>test_quality_agent.py</code>
5	2 0 2 6 - 0 9 - 2 3	Built the test suite <code>test_quality_agent.py</code> (7,853 bytes — the most comprehensive Python test file in the project). Copilot generated the initial 8-test skeleton; remaining 4 tests written manually. Tests cover: zero-rejection low risk, minor/medium/high/critical thresholds, multi-item aggregation, high-percentage rejection flag, corrective-action wording, and the <code>/health</code> endpoint's tool list.	Initial Copilot draft had <code>test_high_rejections_high_risk</code> (lines 71–82) missing all assertions after the <code>response.status_code</code> check — the test body was left incomplete with just <code>data = response.json()</code> and no assert statements. The test passed vacuously (no assertions = always green). Also, three tests used <code>delivery_id: "one"</code> (string) instead of an integer, which Pydantic rejected with a 422 before the endpoint logic ran, making those tests test the wrong thing.	Fixed by writing the missing assertions (<code>assert data["risk_level"] == "High",</code> etc.) and correcting <code>delivery_id</code> to 1. Confirmed 12/12 quality agent tests green via <code>pytest test_quality_agent.py -v</code> .	<code>test_quality_agent.py</code> lines 71–82 (corrected), lines 148–162 (multi-item aggregation), lines 164–176 (HIGH_REJECTION_PERCENTAGE flag)

6	2 0 2 6 - 0 9 - 2 5	Built the React front-end: QualityInspectionsPage.jsx, NonConformancesPage.jsx, QualityRiskPanel.jsx, QualityChecklist.jsx, QualityLifecycleFlow.jsx, NcrRecord.jsx, and qualityFormat.js. Pages were originally a single combined component; split into two separate pages after discussing separation of concerns with Gemini.	Bug 4 (wrong API mock path): When Copilot first generated QualityInspectionsPage.test.jsx, the mock for qualityApi intercepted calls to /quality instead of the actual module path ../services/qualityApi. This caused qualityApi.listInspections to remain unmocked, so every test hit the real (unavailable) backend and failed with network errors rather than the expected assertion failures. Bug 5 (raw JSON in QualityRiskPanel): The initial QualityRiskPanel.jsx rendered {JSON.stringify(analysis)} directly inside the card — the full agent response as a raw JSON blob. It showed no structure, no severity badge, and no readable corrective-action text.	Bug 4 fixed by correcting the vi.mock import path to '../services/qualityApi' (test file line 10). Bug 5 fixed by replacing the raw dump with the structured detail-row layout now in QualityRiskPanel.jsx (lines 23–49), using RISK_TONE from qualityFormat.js and the humanize() helper for PascalCase enum values. Front-end tests: 15 tests across QualityInspectionsPage and NonConformancesPage describe blocks in QualityInspectionsPage.test.jsx.	QualityInspectionsPage.test.jsx lines 10–20 (corrected mock); QualityRiskPanel.jsx lines 27–30 (RISK_TONE badge); qualityFormat.js lines 26–30
7	2 0 2 6 - 0 9 - 2 7	Added qualityFormat.js shared utilities (severity/status/risk tone maps, humanize(), dateTime(), qty(), suggestNextNcrStatus()), finalised the QualityLifecycleFlow.jsx 9-stage material chain, and wrote the integration/E2E verification in verify-component4.ps 1. Asked Copilot to draft the PowerShell verification script.	Copilot's initial script used a static \$inspection.id = 1 instead of reading the id returned from the POST /api/quality-inspections response. This meant the NCR lookup in the subsequent step always searched for NCRs linked to inspection 1, regardless of what was actually created, and the check silently passed even when the NCR had a different inspectionItemId. Also, the evidence assertion checked \$inspection.evidence.Count without wrapping in @(), which returns null for a single-element array in PowerShell rather than 1.	Fixed by replacing the hardcoded id with \$inspection.id from the response (line 59: \$inspection.id -gt 0 -and @(\$inspection.evidence).Count -eq 1), and adding @() coercion throughout. All 5 verify checks now pass (verify-component4.ps1 lines 59–80).	verify-component4.ps1 — 5/5 checks PASS; confirmed output: "Component 4 verification passed: inspection #..., NCR #NCR-..."

8	2 0 2 6 - 0 9 - 2 9	Final review pass: added the canReview role guard in NonConformancesPage.jsx (line 53) so the transition UI is hidden from QualityInspector users, corrected the TypeError crash caused by calling .trim() on undefined resolution (documented in the inline comment at lines 72–75 of NonConformancesPage.jsx), and confirmed the 15 front-end tests and 12 Python agent tests all passed before branch push.	A TypeError: Cannot read properties of undefined (reading 'trim') was thrown at line 75 of NonConformancesPage.jsx when a reviewer clicked "Save transition" without having typed in the Resolution field. The review[ncr.id] spread produced { status: "Resolved" } with no resolution key, so current.resolution.trim() crashed outside the try/catch, leaving the button permanently disabled with no error message shown.	Fixed by coercing: `const resolution = (current.resolution		").trim() before the guard. Confirmed with Vitest test "refuses a resolution-bearing transition with no resolution text" (line 337): the error banner appears and transition NonConformance` is never called.	Non ConformancePage.js line 72–82; QualityInspector specifications Page. test.js line 337 – 351
---	--	--	---	--	--	---	---

3. Reflection

3.1 What AI did well

AI tools accelerated the repetitive structural work that dominates a domain like quality management. Copilot was genuinely useful for generating the initial Pydantic model classes (QualityRiskInput, QualityRiskOutput, NcrRecommendationOutput) — the field names, types, and docstrings were syntactically correct and matched FastAPI's validation conventions without revision. It also produced a useful skeleton for the PowerShell verification script, getting the Invoke-RestMethod call structure, token acquisition function, and Assert-True helper right on the first attempt. Claude was helpful for drafting the NCR auto-raise logic inside the controller — the concept of reading `rejectedQuantity > 0` on each `InspectionItem` and creating a corresponding `NonConformance` was understood and implemented structurally correctly, even if the enum names were wrong. Gemini was useful as a sounding board when deciding to split the original combined `QualityPage` into two separate pages (`QualityInspectionsPage` and `NonConformancesPage`), articulating the separation-of-concerns argument clearly and suggesting the `qualityFormat.js` shared module to avoid duplication across both pages.

The `QualityLifecycleFlow.jsx` component (207 lines, 9 pipeline stages from Material Request through to Resolution) benefited from Copilot's ability to produce the repetitive `STAGES` array entries quickly. The overall architecture of the React risk panel — showing agent name, tool, execution source, risk level badge, and corrective action — was suggested by Copilot and was structurally sound, needing only style corrections.

3.2 Where AI failed or required correction

Type and naming mismatches were the most consistent failure mode. All three tools produced code that used string literals where enum values were required, or numeric types where string labels were expected. The `ncr_severity` field returning lowercase strings ("high") instead of PascalCase enum-compatible values ("High") is representative: the code appeared correct on a surface reading but produced silent data corruption when deserialised in C#. These bugs required careful cross-layer inspection — reading both the Python output and the C# consumer — to find.

AI-generated tests regularly had missing or vacuous assertions. The

`test_high_rejections_high_risk` test body was left at `data = response.json()` with no assertion — a test that can never fail. This is a subtle and dangerous failure: the CI pipeline would have stayed green while the high-rejection path was completely untested. I now review every AI-generated test for the presence of at least one `assert` statement on the response data before accepting it.

Prompt injection risk was introduced, not flagged. Copilot's draft of `quality_agent.py` forwarded `item.rejection_reason` verbatim to the LLM prompt. In a quality management context, rejection reasons may contain supplier names, product codes, or adversarial text. Copilot did not warn about this; I identified the risk independently and resolved it by making the agent deterministic (no LLM call) rather than attempting to sanitise the input. This was the right architectural decision but required recognising the threat model, not just reviewing syntax.

PowerShell-specific behaviour was not known to Copilot. The `@()` coercion needed when a PowerShell object might return a scalar or an array is a subtle language quirk. Copilot's script omitted it, producing a verification check that appeared to pass (returning `$null.Count` is not an error in PowerShell, it just returns nothing) while actually not testing the assertion at all.

The wrong module path in the Vitest mock (`/quality` instead of `../services/qualityApi`) is a class of error Copilot makes frequently: it guesses plausible-looking import paths rather than reading the actual module structure of the project. Every generated test file must have its import paths verified against the actual file tree before trusting it.

3.3 Lessons learned

1. **Verify enum names across language boundaries immediately.** The C#-to-Python boundary is where type mismatches hide. After every session that touches both layers, I now run a quick integration check (or at minimum, compare enum values manually) before committing.
2. **Test test-code, not just production code.** AI-generated unit tests need to be tested: check that each test actually fails when the code is broken. Running `pytest` with the correct assertion deliberately negated is a 30-second check that would have caught the vacuous assertion problem on day one.
3. **Deterministic agents are safer than LLM-backed agents for compliance-critical domains.** For quality management — where NCR data may be used in legal or supplier-contract disputes — a deterministic rule-based agent is more defensible than an LLM one. The original design used an LLM; the final `quality_agent.py` uses no LLM at all, which also removed the prompt injection surface.
4. **Read every generated script in the target language's idiosyncrasies, not just for logical correctness.** PowerShell's scalar-vs-array coercion (`@()`) and JavaScript's `vi.mock` path resolution are language-specific pitfalls that general-purpose AI tools handle poorly because they generalise from the common case.

3.4 What I would do differently

In a future project I would establish a **cross-layer type contract file** (e.g., a shared JSON Schema or OpenAPI spec) before writing any agent code, so that enum value names are agreed between the C# models and the Python Pydantic schemas from the start rather than discovered through runtime failures. I would also require that any AI-generated test file be run with the production code intentionally broken (e.g., by changing a return value) before accepting it, to confirm that the tests actually exercise the logic. Finally, I would define the security threat model for each agent — particularly whether user-supplied text reaches the LLM — as a checklist item at the start of agent development, not as an afterthought discovered during code review.

4. Signed Declaration

I declare that this log accurately represents my use of AI tools during the development of Component 4. All AI-generated code was reviewed, corrected where necessary, and tested before being committed. The bugs documented above are real defects found during development; the fixes described are the ones present in the final codebase. No AI tool was used to write this log.

Signed: Anoja · **Student ID:** IT24XXXXX

Date: 2026-09-30

Component: 4 — Quality Inspection & Non-Conformance Management

Branch: feature/quality-inspection

Actual Codex maintenance disclosure

AI-assisted maintenance record: 2026-10-05

Tool: OpenAI Codex. Exact model identifier was not recorded in this maintenance log.

Scope: user-requested React/Flutter parity, seven-role workspaces, material validation, procurement approval/PO synchronization, inspection evidence and NCR validation, audit/activity presentation, email failure handling, documentation and GitHub publication.

Outputs: implementation changes, regression tests, local browser review script, responsive Flutter shell, bundled open-license Plus Jakarta Sans, updated README/compliance matrix/report draft and safer source packaging.

Corrections: fixed layout overflow and navigation tests after taller dashboard cards; preserved the selected quotation request; corrected misleading officer/manager labels; restricted Site Officer requests to the own-request endpoint; removed unauthorized React supplier prefetch for read-only users. Browser screenshots and tests were used to verify behavior. The sandboxed backend test run initially failed on Windows EventLog permissions; the authorized rerun passed 307 tests.

Verification actually observed: API 307, Python 28, React 163, Flutter 149 tests passed; final authentication/shell follow-up 10 passed; production browser builds passed; React lint exited successfully with warnings. Full role review 179 checks, followed by 44 passing targeted browser checks after the discovered React permissions defect was fixed.

Limitations: no fabricated student signatures, reflections, commit history, CI/deployment results, SMTP delivery, physical-device demonstration or video evidence. Team member review and understanding of this revision remain to be confirmed by the students. Existing individual logs and historical declarations must be reviewed by their authors; this document does not sign on their behalf.

Students should add their own verified involvement to their individual AI logs. The supplied assignment prohibits external AI assistance during the final demonstration/viva and requires student-authored individual reflections.

Later 5 October work: refined only mobile presentation, added actionable administrator field validation, and verified 13 existing navigation/material-request tests plus the new administrator regression and clean Flutter analysis. Built a release-mode Android APK from source 3b94c1e and published it in repository commit 23d5f22. Release-asset uploads failed due to network timeout/reset; the direct repository APK push succeeded. The APK uses the emulator API URL and existing development signing key; physical-device behavior remains unverified.

Consolidated-report revision: read the supplied 17-page PDF with pypdf, expanded four component evidence sections from actual repository paths and Git history, embedded existing individual logs with an unverified-history notice, and added environment/startup/APK/deployment completion guidance. Personal reflections and signatures were not generated as student-authored evidence. No application source changed during this documentation revision.

ADR appendix: 0001-quotation-analysis-agent-and-workflow-state.md

ADR-001 — Quotation Analysis Agent runtime and procurement workflow state

Status: Accepted · **Date:** 2026-09-19 · **Owner:** Component 2 (Supplier, Quotation & Procurement Management), SLIIT IT24102414

Scope: spec §4 (supplier selection), §5 (deterministic rules), §10 (agent) · **Evidence:** commits abdc6d5, 38bbca9, 6161ecc on feature/supplier-procurement

Context

Component 2 must turn an **approved** material request into an auditable supplier recommendation that a Procurement Manager approves **before** any purchase order exists. Three forces shaped the design:

1. **Determinism is a hard requirement.** Spec §5 lists ten business rules that ASP.NET Core must enforce as a gate; the agent may only recommend.
2. **A real agent layer is required** (SE3090): planning, tool use, structured JSON output, observable structured summaries only — no chain-of-thought.
3. **The workflow pauses on a human.** Between "recommendation produced" and "manager decides" the process idles for an unbounded time while other requests keep being processed, and every step must be reconstructable for audit.

Options considered

#	Option	Verdict
A	ASP.NET Core calls an LLM directly; the LLM picks the winner	Rejected — a non-deterministic, networked component on the critical path, untestable without provider mocking, and it would let a model decide an award.
B	Rule-based filter/rank only in C# (no agent service)	Rejected as the primary path (fails the agentic-AI requirement) but kept as the degraded path when the agent service is unreachable.
C	Python FastAPI agent microservice exposing allow-listed deterministic tools + optional LLM rationale; the C# API re-validates everything	Chosen.
D	LangGraph-style multi-agent orchestration framework	Rejected — a two-tool decision with a human-in-the-loop pause does not need a graph runtime or a checkpoint; here the pause is a database row, not a graph checkpoint.

Decision

1. **Runtime.** The agent runs as its own Python FastAPI service (backend/agent_service/quotation_agent.py), addressed by AgentService:Ur1 (http://127.0.0.1:8001) through /health and /analyze, called by QuotationAgentClient under a 10-second cancellation budget.

2. **Tools are allow-listed and role-separated.** The Python quotation agent exposes only `{filter_eligible, rank_by_total}`. The distinct C# `ProcurementPlanningAgent` exposes only four read tools: `GetMaterialRequest`, `GetMaterialDetails`, `GetProjectDetails`, and `GetAvailableQuotations`. It cannot approve procurement, issue a purchase order, or modify supplier records.
3. **Planning is a structured persisted contract.** `ProcurementPlanningAgentService` grounds the objective and input facts in PostgreSQL, produces ordered delegated steps, required deterministic checks and risk flags, and records summaries for all four allow-listed tools in `agent_workflow_steps.structured_result`. Only observable summaries—not hidden reasoning—are stored.
4. **The winner is decided deterministically.** Eligible = Active supplier **and** unexpired quotation. Full-coverage quotations rank first by `total_amount` ascending; a partially covering quotation can never be awarded as the sole winner — it is surfaced as a warning during comparison/enforcement.
5. **The LLM explains; it does not decide.** With `ANTHROPIC_API_KEY` set, Claude rewrites only the rationale sentence (2–3 sentences, sanitized, ≤600 chars); any failure falls back to the deterministic template rationale. A default checkout runs with **no API key and no external call**.
6. **C# never trusts the agent.** `ProcurementValidationAgent` independently checks positive quantities, Active supplier, unexpired quotation, real quotation/supplier identity, material coverage, total reconciliation, promised-delivery validity, required structured fields, and manager approval before PO authorization. Its persisted output is `{ valid, errors, warnings }`; failures move the workflow to `RevisionRequired`, never `AwaitingApproval`.
7. **Failure is contained, never silent.** Technical exceptions → workflow `Failed`; unsafe or invalid business output → `RevisionRequired` with auditable validation results. In both cases no purchase order can be created.

Workflow state persistence (PostgreSQL)

Table	Role in the workflow
<code>agent_workflows</code>	One row per run: <code>material_request_id</code> , <code>initiated_by_user_id</code> , <code>objective</code> , <code>status</code> , <code>approval_status</code> , <code>final_outcome</code> , <code>started_at</code> / <code>completed_at</code> . Survives process restarts.
<code>agent_workflow_steps</code>	Append-only, ordered audit trail: <code>agent_role</code> , <code>step_name</code> , <code>step_order</code> , <code>status</code> , <code>structured_result</code> (jsonb), <code>validation_result</code> (jsonb), <code>error_message</code> , <code>timestamps</code> .
<code>agent_approvals</code>	Append-only human decision: <code>reviewed_by_user_id</code> , <code>decision</code> (Approve / Reject / RevisionRequested), <code>comment</code> , <code>decision_date</code> .

- **Why the database, not memory.** The workflow is paused for an indefinite human window and must be re-readable later through `GET /api/procurement-workflow/{id}/history`; an in-memory queue or background scheduler would lose both the paused state and the audit trail.
- **Step isolation.** Planning (`ProcurementPlanningAgent`), analysis (`QuotationSupplierAnalysisAgent`) and validation (`ProcurementValidationAgent`) are separate rows moving `Running` → `Completed/Failed`, so a failure is attributable to a specific step.
- **Status model.** `Pending` → `Running` → `AwaitingApproval` → `Completed` | `Failed`; `RevisionRequested` returns the workflow to `Pending` for another pass instead of creating a PO.
- **Atomicity.** Purchase-order creation validates preconditions, reads the winning recommendation from the step's `structured_result`, then writes `purchase_orders` + `purchase_order_items` and repoints

quotation statuses (winner Selected, losers Rejected) inside one explicit transaction whenever the provider is relational (the in-memory test provider supports no transactions).

- **Idempotency and the approval gate.** `ValidatePurchaseOrderCreationAsync` blocks a second non-cancelled PO for the same request (§5.10) and blocks creation when no Approved `agent_approvals` row exists (§5.8).

- **Schema ownership.** EF Core migrations stay the single source of truth; the jsonb columns are mapped to string properties on the entities.

Consequences

Positive — decisions remain deterministic and unit-testable (15 xUnit tests, including a full replay of the cement scenario); an agent outage degrades instead of blocking; LLM usage is one short sentence per run, or zero by default; per-step audit trail; the cross-language JSON boundary is validated in exactly one place.

Negative / accepted — two runtimes to launch locally (mitigated by the in-process fallback and a dedicated CI job for `agent_service`); the `snake_case` ↔ `PascalCase` boundary did bite once, because `AgentRecommendationDto` is `PascalCase` while the agent emits `snake_case`, so every field bound to null (fixed in 6161ecc with `JsonNamingPolicy.SnakeCaseLower` scoped to that read); rationale text is model-generated, therefore sanitized and never used for decisions; `agent_workflows` / `agent_workflow_steps` / `agent_approvals` are shared with the other components, so column changes need team agreement.

References

- `docs/component2_spec.md` §5.7 (schema validation), §5.8 (approval gate), §5.10 (idempotency), §10 (agent contract)

- Tests: `ProcurementValidationServiceTests` (8), `ScenarioReplayTests` (1), `AuthServiceTests` (6) → 15 xUnit; `backend/agent_service/test_quotation_agent.py` (9); React vitest (15)

- Agent contract tests that caught the two agent defects:

```
test_invalid_quotation_is_not_ranked_or_recommended,
test_fallback_does_not_resurrect_ineligible_suppliers
```

ADR appendix: ADR-002-component2-agent-architecture.md

ADR-002 — Component 2: Agentic Quotation Analysis Architecture

Status	Accepted
Date	2026-09-19
Deciders	IT24102414 (Supplier, Quotation & Procurement Management)
Scope	Component 2 — Supplier, Quotation & Procurement Management
Related	docs/component2_spec.md §4.5, §5, §7, §10, §11 · commits 38bbca9, 6161ecc
Supersedes	n/a *(first ADR recorded for this component — no ADR-001 exists in docs/)*

1. Context & problem statement

SE3090 requires an Agentic AI contribution to procurement: analysing supplier quotations and recommending an award. The tension is that procurement decisions are commercially sensitive while the AI is advisory. Two failure modes had to be designed out:

1. **The AI overriding deterministic business rules** — e.g. recommending a suspended supplier because it is cheapest.
2. **Silent, unauditable AI output** — a recommendation that reaches a purchase order without an evidence trail or a human decision.

The quotation set must also be re-evaluated against live data: a supplier can be suspended *after* the AI has run, so the AI's answer can be stale.

2. Decision

D1 — A separate Python FastAPI microservice for the analysis agent.

backend/agent_service/quotation_agent.py (FastAPI, uvicorn on 127.0.0.1:8001) exposes GET /health and POST /analyze. Python was chosen over extending the C# API because the agentic layer is exploratory by nature (tool selection, prompting, LLM rationale) and is expected to iterate independently of the stable, rule-governed API. It also keeps AI dependencies out of the .NET deployable, and lets the agent be scaled, restarted or replaced without touching procurement logic.

D2 — C# ASP.NET Core remains the single source of truth.

ProcurementValidationService.cs independently re-derives eligibility (§5.1–5.10)

and re-validates the agent's answer. The agent is advisory; it cannot write to `purchase_orders`, cannot approve anything, and cannot advance a workflow past `AwaitingApproval`. The schema gate (§5.7) rejects any payload that does not match the §10 contract before it is stored.

D3 — Two-phase HTTP with an in-process fallback.

`QuotationAgentClient` calls the agent with a 10 s budget (`cts.CancelAfter(10s)`, `HttpClient` timeout 12 s). On non-success or unreachability it executes `ExecuteFallbackAnalysis`, an in-process mirror of the same deterministic rules, so procurement continues when the microservice is down.

D4 — Workflow state persisted in PostgreSQL, shared table structure.

Execution state lives in `agent_workflows`, `agent_workflow_steps` and `agent_approvals` — deliberately **shared** across all four components' agents per the ERD change-control note. Three ordered steps are recorded: `ProcurementPlanningAgent` → `QuotationSupplierAnalysisAgent` → `ProcurementValidationAgent`. Each step keeps `structured_result` / `validation_result` (jsonb), `status` and `error_message`. Only observable structured summaries are stored — no raw model chain-of-thought. Status transitions are `Pending` → `Running` → `AwaitingApproval` → `Completed` | `Failed`.

D5 — Human-in-the-loop approval gate.

`POST /api/procurement-workflow/{id}/decision` (Manager/Administrator only) records an `agent_approvals` row. Rule §5.8 blocks purchase-order creation until an `Approved` row exists; §5.10 blocks duplicate POs; §5.9 locks the source quotation and request once the PO exists.

3. Alternatives considered

Alternative	Why rejected
Agent logic inside the C# API	Couples AI iteration cadence to the stable API; agent cannot be run or scaled independently
LLM with tool-use autonomy (agent decides and acts)	Violates §10 "what it never does"; untestable against §5; risks selecting a suspended/expired supplier
Storing full model output / chain-of-thought	Unnecessary, non-auditable, prompt-injection surface
Fire-and-forget agent call	No evidence trail; §7/§11 require step-by-step history
Using the in-process fallback as the only implementation	Loses the independent cross-check that the C# and Python layers agree

4. Consequences

Positive

- Two independent implementations of the same rules cross-check each other; a divergence surfaces as a validation failure instead of a bad purchase order.
- Every recommendation is reproducible and auditable from `agent_workflow_steps`.
- Zero silent purchase orders: \$5.8/\$5.10 make a bad path fail loudly.
- The agent is independently testable (pytest) and deployable.

Negative / accepted risks

- A second runtime (Python 3.12, uvicorn) must be running; a dead agent silently degrades to the fallback rather than erroring.
- **Contract drift is the main risk class.** The C#→Python request uses `snake_case` property names; the Python→C# response originally deserialized into a PascalCase DTO with no mapping, silently producing an all-null recommendation that then failed schema validation. Fixed in 6161ecc by reading the response with `JsonNamingPolicy.SnakeCaseLower`, scoped to that read only.
- `Program.cs` originally defaulted `AgentService:Url` to port **8000** while `appsettings.json` set **8001** — a mismatch that would have silently skipped the agent and used the in-process fallback. Now aligned on **8001** in both.
- Notification e-mails are best-effort and wrapped so a failure never rolls back the transaction they report on.

5. Verification evidence

Check	Result
C# tests (dotnet test backend/BuildWise.Api.Tests)	20 passed, 0 failed — includes agent-boundary schema tests
Python agent tests (pytest test_quotation_agent.py)	9 passed, 0 failed
React tests (npx vitest run)	4 files, 15 passed
npm run build	exit 0, 58 modules
End-to-end replay (spec §11, request #1)	workflow #51 → AwaitingApproval → Approved → purchase_orders row (250 bags @ 2 100.00 = 525 000.00); quotations #3 Selected, #6/#7 Rejected

CI (`.github/workflows/ci.yml`) runs the backend, agent, web and mobile suites on push/PR to main.

6. Known gaps

- An HTTP 400 was observed on the PO-creation call that nonetheless persisted a valid PO. Not reproducible on a clean run (the identical request returned 200 OK with the correct payload). Suspected EF Core tracking conflict when `CreateFromWorkflow` re-enters `GetById` on the same scoped `DbContext`. Open, non-blocking.
- `component2_spec.md` §10's response schema omits `recommended_supplier_name`, which the implementation (and the React UI) do include.
- A quotation marked `valid = false` was previously ranked and could win; fixed in 38bbca9 (skip + strict fallback screening), with regression tests.

ADR appendix: ADR-003-react-state-management.md

ADR-003 — React State Management Strategy

Status: Accepted · **Date:** 2026-09-23 · **Owner:** Component 2 (IT24102414), agreed by all component owners

Scope: SE3090 spec §7 (React), §9 (cross-platform workflow)

Context

BuildWise's React front-end serves five role-based dashboards (Admin, Procurement Manager, Site Engineer, Delivery Staff, Quality Inspector). Each feature area has independent data lifecycles:

- **Component 1** — material requests, approval queues
- **Component 2** — suppliers, quotations, RFQs, procurement workflows, agent analysis
- **Component 3** — deliveries, receiving records
- **Component 4** — quality inspections, non-conformances

The spec requires: routing with protected routes by role, client-side validation, API error handling, loading/empty states, and testability without a running backend.

Three cross-cutting concerns shaped the choice:

1. **Auth context** — JWT, role, and logout must be available everywhere without prop drilling.
2. **Feature isolation** — each component team owns its feature folder; state solutions must not create hard cross-team coupling.
3. **Test simplicity** — vitest + React Testing Library must be able to test components without complex store setup.

Options considered

#	Option	Verdict
A	React Context + useState/useReducer per feature	Chosen — enough for SE3090 scale, testable, no extra dependencies
B	Redux Toolkit	Rejected — significant boilerplate; adds a required peer dependency; overkill for assignment scale
C	Zustand	Rejected — not a React built-in; adds a dependency; team unfamiliar
D	React Query / TanStack Query	Rejected — good for server state caching but adds complexity beyond spec requirements
E	Single global Context for all state	Rejected — causes unnecessary re-renders across all four feature areas on any state change

Decision

1. **Auth is a single global AuthContext** (`src/auth/AuthContext.jsx`) providing `user`, `token`, `login`, `logout`. All protected routes read from this context. No auth state is duplicated in feature components.
2. **RBAC guard** is a separate `accessControl.js` module with a `canAccess(role, resource)` function, keeping permission logic testable independently of components.
3. **Feature-local state** uses `useState` and `useReducer` within feature components. No cross-feature shared state exists — components communicate through the API, not through shared client-side state.
4. **API calls** use thin fetch wrappers in `src/services/*.js` (e.g., `procurementApi.js`, `qualityApi.js`). Components call these services in `useEffect`; `loading/error/data` are local `useState` variables.
5. **No external state library** is added. The single `react-router-dom` dependency handles routing; all other state is native React.

Workflow state (read-only in React)

The agentic AI workflow state (`agent_workflows`, `agent_workflow_steps`, `agent_approvals`) is **server-owned and server-persisted**. React only reads it via GET endpoints and displays it. No client-side caching of workflow state was needed.

Consequences

Positive

- Zero extra state library dependencies — consistent with the existing `package.json`
- Every component is independently unit-testable with a simple mock `AuthContext` wrapper
- Feature isolation: Component 2's procurement state does not affect Component 4's quality state
- Vitest runs pass with simple `render(<Component />)` patterns

Negative / accepted

- No built-in cache invalidation — components re-fetch on mount; accepted for assignment scale
- Large list pages do not paginate on the client — pagination is a server-side API parameter

References

- `web/buildwise-web/src/auth/AuthContext.jsx`
- `web/buildwise-web/src/auth/accessControl.js`
- `web/buildwise-web/src/services/procurementApi.js`
- React Testing Library tests: `src/Features/procurement/components/*.test.jsx`

ADR appendix: ADR-004-flutter-state-management.md

> Scope update (2026-10-05): all seven internal roles now use Flutter. Role-filtered navigation matches React, including procurement and administration. Per-screen StatefulWidget state and injected domain services remain the selected approach; camera/gallery evidence and local notifications are implemented. The older three-role restriction below is historical, not the current behavior.

ADR-004 — Flutter State Management Strategy

Status: Accepted · **Date:** 2026-09-23 · **Owner:** All component owners (Flutter is shared)

Scope: SE3090 spec §8 (Flutter mobile), §9 (device feature, secure storage, API integration)

Context

BuildWise's Flutter application runs on Android (primary submission target: APK) and optionally on Windows desktop and Chrome (web). It must:

1. Authenticate against the BuildWise ASP.NET Core API and store the JWT securely on-device.
2. Navigate role-conditionally to field feature areas. Historical scope: the mobile app was a **field operations tool for three roles** — Site Engineer (material requests), Site Officer (delivery receiving) and Quality Inspector (inspections, NCRs). Procurement, management and administration run on the React web app. Suppliers are external stakeholders contacted by email and have no mobile surface at all.
3. Display live procurement workflow status with **local push notifications** when status changes (device feature — spec §9.2).
4. Support login → authenticated workflow → logout without leaking tokens.

The assignment scale is a team of four working independently on feature folders. Flutter state management must not create hard coupling between feature teams.

Options considered

#	Option	Verdict
A	StatefulWidget + setState per screen, shared AuthService singleton	Chosen — simplest, testable, no extra packages needed
B	Provider / Riverpod	Rejected — adds dependency; team unfamiliar; overkill for four screens
C	BLoC / Cubit	Rejected — heavy boilerplate; difficult to learn quickly mid-sprint
D	GetX	Rejected — opinionated, non-idiomatic, conflicts with Flutter lint rules in <code>analysis_options.yaml</code>
E	InheritedWidget (manual)	Rejected — lower-level than needed; Provider wraps this already

Decision

1. **AuthService** (`lib/core/auth/`) is a singleton holding the JWT, parsed user role, and base API URL. It wraps `flutter_secure_storage` for encrypted on-device token storage. All screens and services read the token from `AuthService`; none store it locally.
2. **Per-screen StatefulWidget + setState** for loading, error, and data state. Each screen owns its own state; no global state store is used.
3. **NotificationService** (`lib/features/procurement/services/notification_service.dart`) wraps `flutter_local_notifications` to display local push notifications when procurement status changes. This is the required **device feature**: on-device notification scheduling without a remote push server, satisfying spec §9.2 without requiring a Firebase project.
4. **ProcurementStatusPoller** (`lib/features/procurement/services/procurement_status_poller.dart`) polls the API on a timer and fires `NotificationService` when status transitions are detected. This is the only background-style process; it runs within the app lifecycle.
5. **Navigation** uses Flutter's built-in `Navigator` with named routes defined in `lib/routes/`. Role-based screen access is enforced in the route resolution logic by reading `AuthService.currentRole`.
6. **No external state management package** beyond the three packages already in `pubspec.yaml` (`http`, `flutter_secure_storage`, `flutter_local_notifications`).

Device feature detail

```

ProcurementStatusPoller (Timer.periodic)

    ↓

GET /api/procurement-workflow/{id}

    ↓

Status changed? —No→ (wait)

    ↓ Yes

NotificationService.showStatusUpdate(title, body)

    ↓

flutter_local_notifications → System notification tray

```

This satisfies the spec requirement for a meaningful device feature (local notifications) without any Firebase dependency, matching the team's decision to avoid a Firebase project for local assignment submission.

Consequences

Positive

- No extra dependencies — `pubspec.yaml` stays minimal and auditable
- `flutter test` works without mocking a state container
- `AuthService` is the single source of truth for the token; no risk of stale tokens in multiple stores
- Notification feature works offline (local, no remote push server)

Negative / accepted

- `setState` causes full-screen rebuilds on data change; acceptable at assignment scale
- Polling instead of WebSocket for status changes; polling interval is configurable
- No persistent offline cache — app requires network for all data; accepted

References

- `mobile/buildwise_mobile/lib/core/auth/`
- `mobile/buildwise_mobile/lib/features/procurement/services/notification_service.dart`
- `mobile/buildwise_mobile/lib/features/procurement/services/procurement_status_poller.dart`
- `mobile/buildwise_mobile/pubspec.yaml`
- Spec §8 (Flutter), §9.2 (device feature)

ADR appendix: ADR-005-deployment-architecture.md

> Deployment status update (2026-10-05): this document records the proposed hosting design, not evidence of a live deployment. No public deployment URLs were verified in the maintenance run. Historical provider free-tier/cost assumptions below must be checked by the team before choosing a current plan. The CI build now takes VITE_API_BASE_URL from a GitHub repository variable, rather than assuming an unverified hosted API URL.

ADR-005 — Deployment Architecture

Status: Accepted · **Date:** 2026-09-23 · **Owner:** Group (all component owners)

Scope: SE3090 spec §11 (deployment), §12 (health, Swagger, public URLs)

Context

The SE3090 spec requires:

- ASP.NET Core API deployed with a public HTTPS URL
- Working /health endpoint
- Working /swagger endpoint
- PostgreSQL deployed securely (credentials via environment variables, not committed)
- React web app deployed with a public HTTPS URL
- Flutter Android APK available for download or sideload
- Agentic AI agent services accessible to the deployed API

Four forces shaped the choice:

1. **Cost** — student project; free tier is a hard requirement.
2. **Simplicity** — no DevOps experience on the team; managed platforms preferred.
3. **PostgreSQL support** — managed Postgres must be available on the same platform.
4. **Environment variable support** — no secrets in code or config files.

Options considered

#	Option	Verdict
A	Railway (API + PostgreSQL + Python services) + Vercel (React)	Chosen — free tier sufficient, managed Postgres included, GitHub-connected deploys
B	Render (API + PostgreSQL) + Netlify (React)	Viable alternative; rejected because Railway's Dockerfile-free deploy is simpler for .NET
C	Azure App Service + Azure Database for PostgreSQL	Rejected — free tier expires after 12 months; credit card required
D	Heroku + Heroku Postgres	Rejected — free dynos removed; costs money
E	AWS Elastic Beanstalk + RDS	Rejected — complex IAM setup; exceeds team's DevOps capacity
F	VPS (e.g., DigitalOcean Droplet)	Rejected — requires manual nginx/SSL setup; maintenance burden

Decision

API — Railway

- ASP.NET Core 8 deployed as a Railway service connected to the GitHub repository.
- `railway.json` (or `Dockerfile` if required) at repository root.
- Environment variables set in Railway dashboard (never in code):
- `ConnectionStrings__DefaultConnection` — Railway-provided PostgreSQL URL
- `Jwt__Key` — signing key
- `AgentService__Url` / `AgentService__RequestUrl` / `AgentService__DeliveryUrl` / `AgentService__QualityUrl` — Railway internal URLs for agent services

PostgreSQL — Railway Managed Postgres

- Railway-provisioned PostgreSQL database.
- EF Core migrations run on startup (`app.MigrateAsync()` in `Program.cs`).
- Credentials injected via `ConnectionStrings__DefaultConnection` environment variable.

Python Agent Services — Railway (4 services)

- Four Railway services (`quotation_agent.py`, `request_agent.py`, `delivery_agent.py`, `quality_agent.py`) each exposed on internal Railway URLs.
- SMTP — configured via `Smtp__Host` / `Smtp__Username` / `Smtp__Password` environment variables; console-logged if `Smtp__Host` is empty.

React — Vercel

- `web/buildwise-web` deployed to Vercel via GitHub integration.
- `VITE_API_BASE_URL` set to the Railway API URL in Vercel environment variables.
- Build command: `npm run build`. Output directory: `dist`.

Flutter APK

- Built locally with `flutter build apk --release`.
- APK (`build/app/outputs/flutter-apk/app-release.apk`) submitted as a binary artifact alongside the repository.
- API base URL in `lib/core/api/api_client.dart` is configurable at build time via a `--dart-define=API_BASE_URL=<url>` flag.

Public URLs (to be populated after deployment)

Service	URL
API	https://buildwise-api.railway.app
Swagger	https://buildwise-api.railway.app/swagger
Health	https://buildwise-api.railway.app/health
React	https://buildwise-web.vercel.app
APK	Submitted as binary with report

Security

- All secrets via environment variables; `.env` is `.gitignored`.
- HTTPS enforced by Railway and Vercel (automatic TLS).
- JWT audience and issuer validated on every request.
- CORS configured to allow only the Vercel origin in production.
- Database credentials never committed.

Consequences

Positive

- Zero infrastructure cost (free tiers)
- GitHub-connected: push to main triggers automatic redeploy
- Managed Postgres: no DBA work, automatic backups
- Vercel CDN: React assets served from edge nodes

Negative / accepted

- Railway free tier has cold-start latency (~5–10 s) after inactivity; acceptable for demonstration
- Python agent services on Railway free tier sleep after 10 min; first agent call after inactivity may timeout (mitigated by the C# in-process fallback in `QuotationAgentClient`)
- APK must be rebuilt and re-submitted manually if the API URL changes

References

- `backend/BuildWise.Api/Program.cs` — `app.MigrateAsync()`, health endpoint registration
- `backend/BuildWise.Api/appsettings.json` — configuration key names (values via env vars)
- `.env.example` — full list of required environment variables

- `web/buildwise-web/package.json` — build scripts
- `mobile/buildwise_mobile/lib/core/api/api_client.dart` — configurable API base URL

ADR appendix: ADR-006-suppliers-are-external-stakeholders.md

Suppliers are external stakeholders — ADR

Status

Accepted. Supersedes the earlier supplier-portal design.

Context

An earlier iteration of BuildWise modelled a Supplier **role**: a login bound to one supplier record through `User.SupplierId`, a signed `supplier_id` JWT claim, a `/api/supplier-portal/*` API surface, a `SupplierPortalController`, a React portal page and a Flutter portal.

That design was rejected because it does not reflect how construction procurement actually works, and it widened the authentication boundary to parties that are not BuildWise employees.

Decision

A supplier is an external stakeholder, not a BuildWise user.

Suppliers do **not**:

- log into BuildWise, or hold an account, JWT, role or password
- have a dashboard, portal or mobile app
- access the BuildWise database directly
- see other suppliers' quotations or any internal procurement data

Communication is external, by email:

```
BuildWise —RFQ email—► External Supplier
External Supplier —quotation email—► Procurement Officer
Procurement Officer —records quotation—► BuildWise
BuildWise —PO email—► External Supplier
External Supplier —physical delivery—► Site Officer (mobile)
BuildWise —NCR / corrective-action email—► External Supplier
```

What is kept — the business entity

Supplier remains a first-class **business entity**, because procurement still

needs the supplier's contact details to send RFQs, record returned quotations, award purchase orders and raise corrective actions. It is referenced by:

- RfqSupplier — which suppliers an RFQ was emailed to
- Quotation — which supplier submitted the recorded quotation
- PurchaseOrder — which supplier the order was placed with
- NonConformance — which supplier a corrective action concerns

POST /api/material-requests/{id}/quotations is the officer-side endpoint that records a quotation **received by email**. It is gated by Policies.SupplierAdministrationOnly (Procurement Officer / Administrator).

Roles

BuildWise has exactly seven internal roles:

```
SiteEngineer → SiteManager → ProcurementOfficer → ProcurementManager
                → SiteOfficer → QualityInspector → Administrator
```

Retired legacy roles

Two earlier aliases were removed once it was verified they were ****permission subsets**** of the roles that replaced them:

Legacy role	Policies it had	Replaced by	Why it was safe
ReceivingOfficer	InternalStaff, DeliveryParticipants	SiteOfficer	SiteOfficer holds both of those plus SiteOperations, QualityReaders and MaterialRequestReaders — a strict superset
ProjectManager	InternalStaff only	SiteManager	SiteManager holds InternalStaff plus both approval policies, procurement reads and delivery — a strict superset

Because each target was a superset, the migration could only ***widen*** access, never narrow it unexpectedly.

The ConsolidateLegacyRoles migration re-points any account still holding a legacy role before deleting the role rows, and de-duplicates so a user already holding the target role is not double-granted. The redundant receiving.officer@buildwise.demo demo login is removed; site.officer@buildwise.demo is the Site Officer account.

The Flutter shell keeps a short-lived legacyAliases map in buildwise_roles.dart purely so a JWT minted ***before*** the migration resolves to the right role instead of locking its holder out. It can be deleted once every client has re-authenticated.

Client split

Client	Roles	Surfaces
React web	all seven	Procurement, approvals, administration
Flutter mobile	Site Engineer, Site Officer, Quality Inspector	Field operations only

Procurement roles signing in on mobile are shown a web-only explainer rather than a partial procurement UI. This keeps purchase orders raisable in exactly one place.

Consequences

Accepted trade-off: a supplier cannot self-serve a quotation. Every quotation is transcribed by the Procurement Officer. That is manual work, but it is the real process, and it keeps authentication limited to internal staff.

Enforced by tests: a token carrying a legacy Supplier role is refused by every endpoint (`RbacAuthorizationTests.cs`), the role is not seeded, and no user row is bound to a supplier.

Migration

Two migrations were applied to the development database:

20261002104811_RemoveSupplierUserAccount — deletes legacy supplier login rows and the Supplier role, then drops the `users.SupplierId` foreign key. It **never** touches the `suppliers` table, so all supplier business records and every RFQ / quotation / purchase-order relationship are preserved.

20261002113029_ConsolidateLegacyRoles — re-points accounts holding a retired role at their canonical replacement (de-duplicated), deletes the now-unused demo login, and removes the ProjectManager, ReceivingOfficer and Supplier role rows.

Verified after applying: roles holds exactly seven rows, no `users.SupplierId` column remains, and supplier / RFQ / quotation / purchase-order counts are intact.

Measured performance appendix

```
{
  "measuredAtUtc": "2026-10-05T09:30:52.975Z",
  "environment": "Local Windows development stack; not a production SLA",
  "requestsPerScenario": 30,
  "concurrencyPerScenario": 5,
  "results": [
    {
      "name": "Shared ASP.NET Core API + PostgreSQL",
      "total": 30,
      "concurrency": 5,
      "succeeded": 30,
      "failed": 0,
      "successRatePct": 100,
      "throughputRps": 267.7,
      "latencyMs": {
        "average": 17.59,
        "p50": 12.86,
        "p95": 78.89,
        "max": 79.83
      },
      "localAcceptanceP95Ms": 2000,
      "passed": true,
      "sampleErrors": []
    },
    {
      "name": "Agent 8001 quotation analysis",
      "total": 30,
      "concurrency": 5,
      "succeeded": 30,
      "failed": 0,
```

```

    "successRatePct": 100,

    "throughputRps": 2.98,

    "latencyMs": {

        "average": 1625.93,

        "p50": 817.64,

        "p95": 6782.05,

        "max": 8522.86

    },

    "localAcceptanceP95Ms": 1000,

    "passed": false,

    "sampleErrors": []

},

{

    "name": "Agent 8002 request analysis",

    "total": 30,

    "concurrency": 5,

    "succeeded": 30,

    "failed": 0,

    "successRatePct": 100,

    "throughputRps": 223.05,

    "latencyMs": {

        "average": 21.93,

        "p50": 8.07,

        "p95": 94.49,

        "max": 94.63

    },

    "localAcceptanceP95Ms": 1000,

    "passed": true,

    "sampleErrors": []

},

{

    "name": "Agent 8003 discrepancy analysis",

```

```

    "total": 30,

    "concurrency": 5,

    "succeeded": 30,

    "failed": 0,

    "successRatePct": 100,

    "throughputRps": 256.81,

    "latencyMs": {

        "average": 19.12,

        "p50": 6.9,

        "p95": 82.03,

        "max": 82.29

    },

    "localAcceptanceP95Ms": 1000,

    "passed": true,

    "sampleErrors": []

},

{

    "name": "Agent 8004 quality-risk analysis",

    "total": 30,

    "concurrency": 5,

    "succeeded": 30,

    "failed": 0,

    "successRatePct": 100,

    "throughputRps": 291.31,

    "latencyMs": {

        "average": 16.75,

        "p50": 9.11,

        "p95": 58.02,

        "max": 58.59

    },

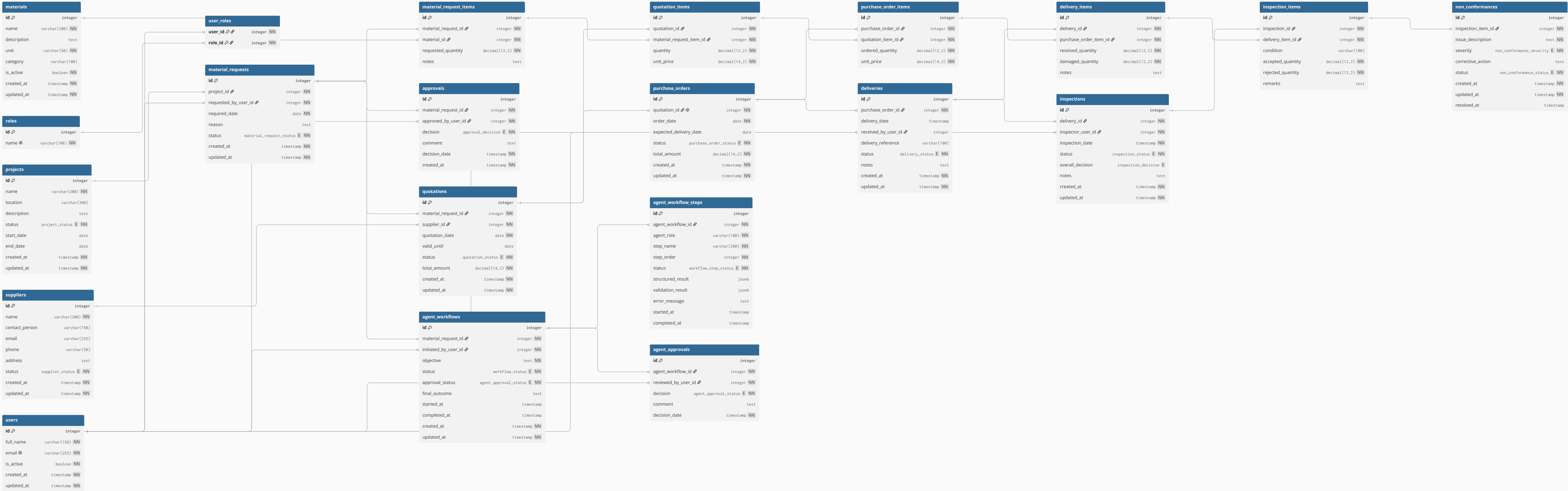
    "localAcceptanceP95Ms": 1000,

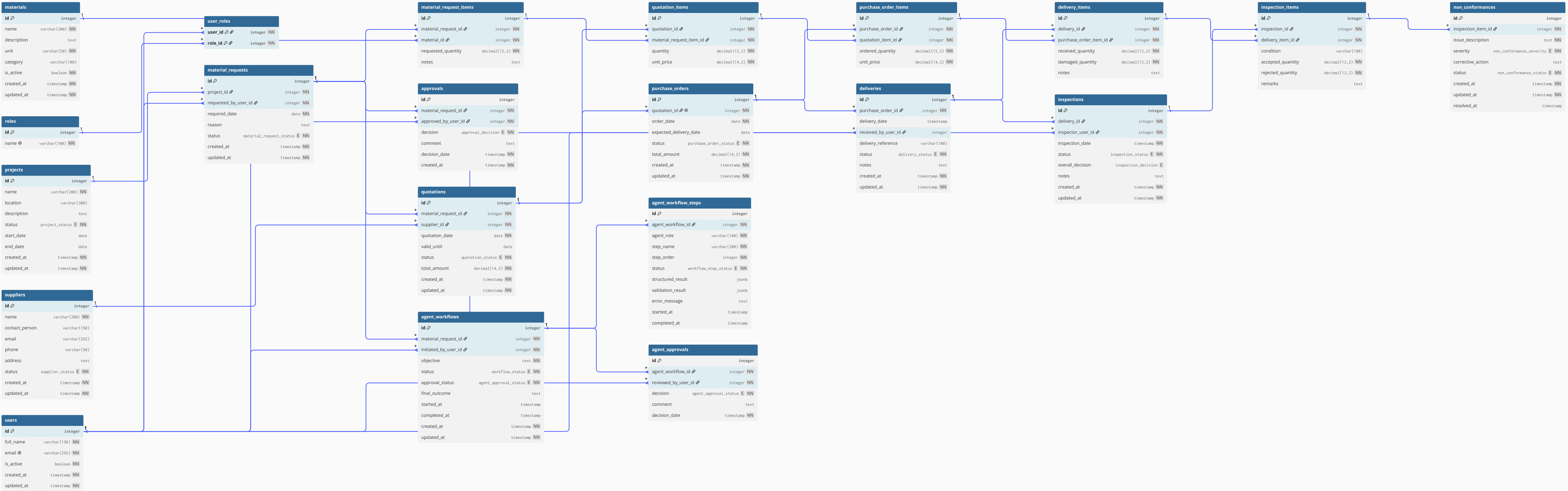
    "passed": true,

    "sampleErrors": []

```

```
}  
]  
}
```





```
Enum project_status {
    Planned
    Active
    Completed
    Cancelled
}
```

```
Enum material_request_status {
    Draft
    PendingApproval
    Approved
    Rejected
}
```

```
Enum approval_decision {
    Approved
    Rejected
    RevisionRequested
}
```

```
Enum supplier_status {
    Active
    Inactive
    Suspended
}
```

```
Enum quotation_status {
    Submitted
    UnderReview
    Selected
    Rejected
    Expired
}
```

```
Enum purchase_order_status {
    Created
    Confirmed
    InProgress
    Completed
    Cancelled
}
```

```
Enum delivery_status {
    Scheduled
    InTransit
    Delivered
    Received
    DiscrepancyReported
}
```

```
Enum inspection_status {
    Pending
    UnderInspection
    Completed
}
```

```
Enum inspection_decision {
    Accepted
    PartiallyAccepted
    Rejected
}
```

```
Enum non_conformance_severity {
    Low
    Medium
    High
    Critical
}
```

```
Enum non_conformance_status {
    Open
    CorrectiveActionRequired
    Resolved
    Closed
}
```

```
Enum workflow_status {
    Pending
    Running
    AwaitingApproval
    Completed
    Failed
    Cancelled
}
```

```
Enum workflow_step_status {
    Pending
    Running
    Completed
    Failed
    Skipped
}
```

```
Enum agent_approval_status {
    Pending
    Approved
    Rejected
    RevisionRequested
}
```

SE3090 ASSIGNMENT 1 | BUILDWISE PROJECT SCENARIO

Construction Materials Procurement, Delivery and Quality Management System

BuildWise - Integrated Full-Stack and Agentic AI Application

TEAM REFERENCE

This document explains the complete BuildWise scenario in one place so every team member can understand the problem, users, four components, shared workflow, four AI contributions, human approvals, web/mobile responsibilities and core data.

Real-world problem

Construction material procurement becomes difficult when material needs start at a construction site but supplier selection and purchasing decisions happen in an office. Requests, quotations, approvals, purchase orders, deliveries and inspections may be tracked through separate calls, messages, spreadsheets and paper records.

BuildWise connects those activities in one system. Site users use a Flutter mobile application for material requests, delivery receiving and quality inspection. Procurement and management staff use a React web application for supplier and quotation management, approvals, purchase orders, monitoring and administration. ASP.NET Core provides the shared secure API and business rules, PostgreSQL stores the shared relational data, and Agentic AI assists decisions while important actions remain under human control.

User roles

- 1. Site Engineer / Site Officer.** Creates material requests, tracks progress and records deliveries from the construction site.
- 2. Procurement Officer.** Maintains suppliers, records quotations, compares options and prepares procurement.
- 3. Procurement Manager.** Reviews request approvals and AI-assisted procurement recommendations, then approves, rejects or requests revision.
- 4. Quality Inspector.** Inspects delivered materials, records accepted/rejected quantities and manages non-conformance cases.
- 5. Administrator.** Manages users, roles and shared system settings.

Implementation note. User authentication and role-based authorization are shared mandatory features and should not be counted as one of the four main business components. Supplier records are business data managed by procurement staff.

Four main business components

Component	Main functionality	Business-specific operation
Material Request & Approval Management	Projects, materials, requests, request items, required dates, reasons, statuses and request approval decisions.	Submit and control a material request, preserve approval history and prevent silent changes after approval.
Supplier, Quotation & Procurement Management	Suppliers, quotation items, validity, comparison, AI-assisted recommendation, procurement approval, purchase orders and purchase-order items.	Evaluate eligible quotations and create a purchase order only after the recommendation passes validation and authorized approval.
Delivery & Material Receiving Management	Expected purchase-order information, deliveries, received quantities, damaged quantities and delivery status.	Reconcile what was ordered against what actually arrived and flag shortages, damage or partial delivery.
Quality Inspection & Non-Conformance Management	Inspections, inspection items, accepted/rejected quantities, quality decisions and non-conformance records.	Complete the inspection and create a traceable non-conformance when rejected material requires corrective action.

Each student owns one component and should implement its ASP.NET Core endpoints, PostgreSQL entities and relationships, React interfaces, Flutter interfaces, tests, documentation, Git evidence and one identifiable Agentic AI contribution.

Example complete scenario

A Site Engineer submits this objective using Flutter:

“Request 250 bags of OPC cement for the Colombo apartment project, required by 22 August for ground-floor column concreting.”

The request is reviewed and approved for procurement. The Procurement Officer then records three supplier quotations:

- **Supplier A:** active, full 250 bags available, valid quotation, total LKR 525,000.
- **Supplier B:** suspended, full 250 bags available, lower total of LKR 510,000.
- **Supplier C:** active and cheapest, but only 200 bags are quoted.

Workflow

1. Flutter submission. The Site Engineer logs in, selects the project and material, enters 250 bags, the required date and reason, and submits the request.

2. ASP.NET Core processing. The API validates the JWT, role, project, material, quantity and date, then saves the request and items in PostgreSQL.

3. Request Validation & Planning Agent. The component-owned agent checks completeness, quantities, dates and urgency, then produces a structured request summary and planning flags. It does not approve the request.

4. Request approval. An authorized manager reviews the request and records Approve, Reject or Revision Requested. Approved requests may continue to procurement.

Workflow continued

5. React procurement preparation. The Procurement Officer records supplier quotations. Each quotation is linked to the original material request, and each quotation item is linked to the corresponding request item.

6. Quotation & Supplier Analysis Agent. This component-owned agent compares eligible quotations using structured data such as supplier status, quotation validity, quantities and calculated totals. It identifies that Supplier B is suspended and Supplier C does not cover the full quantity.

7. Structured recommendation. The agent recommends Supplier A and returns a short structured rationale/evidence summary, ranked alternatives and warnings. BuildWise stores observable summaries and results - not private model chain-of-thought.

Deterministic validation

Before the recommendation can reach procurement approval, ASP.NET Core verifies fixed rules:

- the material request exists and has the required approval;
- the recommended supplier exists and has Active status;
- the recommended quotation exists and is not expired;
- quotation items belong to the requested material items;
- the recommended quantity covers the requested quantity;
- the quotation total matches the quotation-item calculation;
- the recommendation follows the required structured output schema;
- purchase-order creation is blocked until authorized procurement approval exists.

Human approval

8. Procurement Manager review. The workflow becomes **AwaitingApproval**. The Procurement Manager uses React to review the recommended supplier, structured rationale, warnings, validation results and execution summary.

The Procurement Manager can:

- **Approve**
- **Reject**
- **Request Revision**

9. Final procurement execution. If approved, ASP.NET Core records the agent workflow approval and creates the purchase order and purchase-order items from the selected quotation. Purchase orders remain owned by Component 2; Component 3 only consumes approved purchase orders for receiving.

10. Flutter status update. The Site Engineer receives the approved procurement status through the mobile application.

Therefore, the assessed procurement workflow is:

Flutter → ASP.NET Core → PostgreSQL → Agentic AI → React approval → ASP.NET Core / PostgreSQL → Flutter status update

Delivery and quality continuation

BuildWise continues the same traceable procurement record after purchase-order approval:

11. Delivery receiving. When the supplier arrives, the Site Officer opens the approved purchase order in Flutter and records the actual delivery. Example: 250 bags ordered, 240 bags received and 5 of the received bags visibly damaged.

12. Delivery Discrepancy Agent. This component-owned agent compares ordered, received and damaged quantities. It produces a structured discrepancy summary such as shortage = 10 bags and damaged = 5 bags, with warnings for partial or abnormal delivery. The backend independently validates the arithmetic before saving the result.

13. Quality inspection. The Quality Inspector opens the delivery in Flutter, performs the inspection and records accepted/rejected quantities. Example: 235 bags accepted and 5 bags rejected.

14. Quality Risk & Non-Conformance Agent. This component-owned agent analyses the inspection result and existing supplier-quality evidence, then suggests risk flags and an appropriate non-conformance/corrective-action summary when required. The Quality Inspector remains responsible for the recorded inspection decision.

15. Non-conformance. If rejected material requires corrective action, BuildWise creates a non-conformance record linked to the affected inspection item and tracks it until resolved or closed.

Complete BuildWise business flow

Material Request → Request Approval → Quotations → AI-Assisted Procurement Evaluation → Manager Approval → Purchase Order → Delivery → AI Discrepancy Analysis → Quality Inspection → AI Quality-Risk Assistance → Acceptance / Non-Conformance

Four specialized Agentic AI contributions

Owned component	Agent	Responsibility
Component 1 - Material Request & Approval	Request Validation & Planning Agent	Checks completeness, quantities, dates and urgency; creates a structured request plan/flags.
Component 2 - Supplier, Quotation & Procurement	Quotation & Supplier Analysis Agent	Compares eligible quotations and supplier evidence; returns ranked options, structured rationale and warnings.
Component 3 - Delivery & Receiving	Delivery Discrepancy Agent	Compares ordered vs received/damaged quantities and flags shortages, partial deliveries or discrepancies.
Component 4 - Quality Inspection & NCR	Quality Risk & Non-Conformance Agent	Assists with inspection-risk interpretation and prepares structured NCR/corrective-action suggestions.

Shared safety rule: deterministic backend validation is a system-wide control, not a fifth AI agent. Agents may analyse and recommend, but authorization, calculations, state transitions and other high-impact actions are enforced by ASP.NET Core and authorized users.

React web application

The React application is mainly for procurement, management, administration and monitoring:

- material-request review and approval visibility;
- supplier management and supplier status control;
- quotation entry and side-by-side comparison;
- Agentic AI workflow start, status, structured summaries and execution-history views;
- recommended supplier, ranked alternatives, structured rationale/evidence summary, warnings and validation results;
- Procurement Manager Approve / Reject / Request Revision controls;
- purchase-order creation and management;
- delivery discrepancy and quality/non-conformance history;
- user and role administration;
- search, filtering, sorting, pagination, dashboards and reports.

Flutter mobile application

The Flutter application is mainly for construction-site operations:

- registration/login/logout and secure JWT token storage;
- create material requests and select required dates;
- track request and procurement status;
- view approved purchase-order information required for site receiving;
- record delivery quantities, damaged quantities and delivery references;
- perform quality inspections and record accepted/rejected quantities;
- view non-conformance status where relevant;
- receive push notifications for important workflow changes.

Third-party integration

Firebase Cloud Messaging (FCM) can be integrated through ASP.NET Core to send mobile notifications such as request approval, procurement recommendation awaiting manager approval, purchase-order creation, delivery discrepancy and inspection completion/rejection. Firebase credentials must remain server-side and failures must be handled safely.

Core data used by BuildWise

Area	Main entities
Identity	users, roles, user_roles
Material Request	projects, materials, material_requests, material_request_items, approvals
Procurement	suppliers, quotations, quotation_items, purchase_orders, purchase_order_items
Delivery	deliveries, delivery_items
Quality	inspections, inspection_items, non_conformances
Agentic AI	agent_workflows, agent_workflow_steps, agent_approvals